ELA: English Language Assistant

Recursive Linguistic Elements, Multimodal Processing, and Explainable Support for Authentic English Input

Vladyslav Rastvorov

R00274535

This thesis has been submitted in partial fulfillment for the degree of

Bachelor of Science (Honours) in Computer Systems

Faculty of Engineering and Science

Department of Computer Science

Munster Technological University Cork

Research Supervisor:	Dr. Alex Vakaloudis
Implementation Supervisor:	Dr. Nasir Ahmad
Academic Year:	2025–2026
Submission Date:	May 2026
Live system:	https://el-a.uk
Source code:	https://github.com/welsol21/FYP_LLM

Declaration of Authorship

This report, *ELA: English Language Assistant*, is submitted in partial fulfillment of the requirements for the degree of Bachelor of Science (Honours) in Computer Systems at Munster Technological University Cork. I, Vladyslav Rastvorov, declare that the work presented in this thesis is substantially my own except where explicitly stated.

- This work was carried out wholly or mainly while in candidature for the above degree at Munster Technological University Cork.
- Where any part of this thesis has previously been submitted for another qualification, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given.
- I have acknowledged all main sources of help.
- Where the thesis is based on work carried out jointly with others, I have made clear what was done by others and what I contributed myself.

Signed: _____

Date: _____

Abstract

This thesis presents the continued development of the English Language Assistant (ELA), a full-stack language learning system designed to make authentic English input more accessible without simplifying or rewriting the source material. ELA ingests text, audio, and video, then transforms the input into a hierarchical contract of *Recursive Linguistic Elements* (RLE), where each sentence is represented as a structured tree of sentence, phrase, and word nodes enriched with grammatical, pedagogical, and bilingual metadata.

The project implements two interconnected approaches. The first is focused on practical support for learners of English, especially users with intermediate and advanced levels of proficiency. For this purpose, a Progressive Web Application (PWA) was developed, bringing together tools for the analysis of authentic English material, the visualization of linguistic relations within a sentence, and the creation of custom learning materials in text, audio, and video form. Particular attention was given to usability: the interface was designed to be intuitive, visually clear, and convenient for everyday use on mobile devices. Within this practical direction, the system was intended not simply to help users read or listen to English, but to support deeper immersion into its grammatical and linguistic structures, including work with bilingual materials and learning based on real, unsimplified content.

The second approach belongs to the research dimension of the project and is aimed at the scientific analysis of relationships between linguistic and grammatical constructions in English. Its purpose is to treat language not as an unlimited collection of isolated examples, but as a system in which a finite number of recurring structural patterns can be identified. For this, the project uses the RLE contract, where RLE stands for Recursive Linguistic Elements, a formal representation developed specifically for this line of research in the earlier research phase of the project. Through this model, a sentence can be described as an organized system of interrelated elements, making it possible not only to analyse individual linguistic phenomena, but also to investigate broader regularities that underlie English grammar.

This idea received empirical support in a separate experiment on the task of selecting a pedagogical note on the basis of the structure of an English sentence. The results showed that when the input representation preserves the recursive structure of the sentence in the form of an RLE contract, note selection improves even when the classifier itself remains

comparatively simple. This is important because it supports the central hypothesis of the project: the grammatical structure of language, and not only its lexical content, can serve as a learnable and predictive basis for subsequent interpretation.

The final result of this work is a fully functional and reproducible pipeline which takes an English input sentence, constructs a hierarchical JSON representation according to the project's RLE contract, and enriches it with linguistic notes. These notes are, in turn, produced on the basis of transformer-classifier inference and then refined for more natural phrasing by a T5-small language model adapted to this task. The classifier itself is also trained not in a single step but in several consecutive stages, in which the structural representation of the sentence, note classification, and the subsequent generation of its final form are combined into one connected system. In this way, the work presents ELA both as an operational system and as an ongoing research programme.

Acknowledgements

I would like to thank my supervisors, Dr. Alex Vakaloudis and Dr. Nasir Ahmad, for their guidance across both the research and implementation dimensions of this project. I am also grateful to my family for their continuing support throughout the development of ELA, and to Munster Technological University Cork for providing the environment in which this work could take shape.

Contents

List of Figures

List of Tables

List of Abbreviations

ASR	Automatic Speech Recognition
CALL	Computer-Assisted Language Learning
CEFR	Common European Framework of Reference for Languages
FR	Functional Requirement
IPA	International Phonetic Alphabet
LLM	Large Language Model
ML	Machine Learning
NFR	Non-Functional Requirement
NLP	Natural Language Processing
POS	Part of Speech
PWA	Progressive Web Application
RLE	Recursive Linguistic Elements
TAM	Tense, Aspect, and Mood
WER	Word Error Rate

Chapter 1

Introduction

1.1 Motivation

Most digital language-learning systems lower difficulty by reducing or simplifying the source material. This makes early progress easier, but it also distances the learner from real English as it appears in speech, media, and written communication. Authentic input is richer, more motivating, and more representative of actual language use, but it is also harder to process without support. The central motivation of ELA is therefore not to simplify authentic English, but to make it explainable.

1.2 Problem Statement

Many learners of English encounter serious difficulties when working with authentic materials such as articles, lectures, podcasts, and audio or video content. Such materials contain dense vocabulary, complex grammatical constructions, idiomatic expressions, and natural speech patterns that make comprehension difficult even for learners at intermediate and advanced levels. Most existing systems address this problem either by simplifying the content, by training isolated skills, or by providing explanations that are too general and too opaque to reflect the real internal structure of the language. As a result, a clear gap remains between classroom English and the English of real communication. ELA takes a different approach: instead of altering the source material, it makes authentic input more accessible through explainable decomposition into interpretable linguistic units and by adding structured support on top of the original text, audio, or video.

This also defines the scope of the present thesis. Its scope includes the implemented system architecture, the linguistic contract, the multimodal runtime path, the frontend visualization layer, and the experimental line concerned with structural abstraction in the generation of pedagogical notes. Outside the scope of this thesis are large-scale classroom deployment and strong claims about measurable learning gains based on user

studies. This boundary is intentional, because the thesis treats ELA primarily as a research-engineering system rather than as a fully validated educational product.

1.3 Research Questions

This thesis is structured around three research questions:

1. How can authentic text, audio, and video material be transformed into structured representations suitable for pedagogical interpretation without simplifying the original content?
2. What model of representation best captures the relationships between the levels of sentence, phrase, and word while remaining sufficiently stable for validation, storage, visualization, and further computational processing?
3. To what extent can structural abstraction, particularly in the form of templates and placeholders, improve the system's ability to generalize when generating pedagogical notes for new language examples?

1.4 Objectives

The ELA implementation phase is organized around the following goals:

1. Construct a multimodal processing pipeline that takes in text, audio, and video and converts them into structured sentence-level representations.
2. Formalize a hierarchical contract capable of carrying grammatical, pedagogical, and bilingual metadata in a stable and validated form.
3. Provide a learner-facing application that exposes linguistic structure through an interactive visualizer, export flow, and media-oriented workflow.
4. Explore whether abstract structural templates support stronger note-generation generalization than raw lexical notation.

1.5 Contributions

These contributions were the following:

- an improved implementation of ELA as a complete-stack system rather than a prototype, deployed at `https://el-a.uk` and available as open-source at [16];
- the formalization and operationalization of the RLE contract as the core data interface between backend, storage, and frontend;

- a rules-first linguistic pipeline that integrates parsing, phrase construction, TAM enrichment, CEFR estimation, translation, phonetic support, and pedagogical notes;
- a research analysis of the placeholder strategy for note generation through the context–pattern duality;
- a detailed evaluation of the current system covering implemented capabilities, empirical evidence, and unresolved limitations.

Taken together, these results allow the thesis to be understood both as a description of an implemented system and as an investigation of the principles of structural representation in language. The practical contribution of the work lies in the creation of a functioning ELA stack which combines the processing of authentic English material, its structural analysis, and user-facing tools for working with the results of that analysis. The research contribution lies in the further confirmation of the central idea of ELA: each sentence can be decomposed into Recursive Linguistic Elements, that is, into the levels of sentence, phrase, and word, and each such element can be described through abstract grammatical attributes rather than through specific words. This, in turn, allows the system to identify stable structural patterns and to use them when working with new language examples of the same grammatical form.

1.6 Thesis Structure

The remainder of the thesis is organized as follows. Chapter 2 reviews the theoretical and applied fields on which ELA draws, including Computer-Assisted Language Learning, authentic materials, NLP-based analysis, CEFR, generative models, and multimodal processing. Chapter 3 connects the current work to the earlier research phase of the project and shows how ELA developed from a conceptual prototype into an implemented system. Chapter 4 describes the RLE data model and explains the contract that underlies the representation of linguistic structure. Chapter 5 is devoted to the idea of context–pattern duality and to the role of structural abstraction in the generation of pedagogical notes. Chapter 6 presents the architecture and implementation of the system, including the backend, runtime, frontend, and multimodal components. Chapter 7 examines dataset and corpus engineering. Chapter 8 evaluates the system and analyses the results obtained. Chapter 9 discusses the limitations of the work and possible directions for further development. Chapter 10 concludes the thesis. After the main body, additional chapters provide repository-based evidence sources, an index of evaluation artefacts, and examples of templates used in the pipeline.

Chapter 2

Background and Related Work

2.1 Computer-Assisted Language Learning

Computer-Assisted Language Learning (CALL) provides the pedagogical background of ELA. Contemporary systems in this area include adaptation, learner modelling, multimodal content, and mobile forms of delivery. The present project develops in this direction by proposing an extension of existing learning tools and methodologies through the active inclusion of authentic language content in the learning process. At the same time, the emphasis is placed not only on the technological side of the system, but also on learner autonomy and on gradual, guided support while working with the material.

Within this approach, software should not simply replace the learner's own analysis. On the contrary, it should externalize part of that analysis in a form that the learner can see, inspect, and explore. This is especially important in independent learning scenarios, where the system should function not only as an exercise tool, but also as a structured explanatory assistant, or, as it is called in this thesis, an English Learning Assistant (ELA).

2.2 Authentic Materials and Cognitive Support

Authentic materials occupy an important place in language learning because they preserve the vocabulary, grammatical constructions, pragmatics, and stylistic features of real communication. It is through such materials that the learner encounters features of language that are rarely fully present in artificially simplified textbook examples: natural tense shifts, fixed expressions, characteristic linking phrases, and the way speakers actually express their thoughts. At the same time, the very qualities that make authentic material valuable also make it more difficult to process.

This creates a central methodological tension. If authentic material is simplified too aggressively, it loses a significant part of its educational value. If it is left in its

original form without additional support, it may become too difficult even for a motivated learner. ELA addresses precisely this problem: the system does not rewrite the original material, but instead adds a layer of structured and explainable support to it. In this sense, the project stands at the intersection of authenticity, pedagogical support, and transparency, since it aims to preserve the original utterance while making its internal structure accessible for study.

2.3 NLP Parsing and Structured Analysis

ELA uses modern NLP tools to extract syntactic and morphological information from English text, including tokenization, lemmatization, part-of-speech tagging, and syntactic parsing. However, such outputs cannot by themselves serve as a convenient representation for the learner. Raw parser output depends on the specific implementation, may be unstable, and in this form is poorly suited for direct use in an educational interface.

For this reason, parsing in ELA is treated not as a final result but as a source of structural data for further interpretation. The resulting signals are converted into a controlled hierarchical representation that can then be used for enrichment, validation, storage, and visualization. This is an important distinction of the project: ELA is not simply a wrapper around an existing NLP library, but a system that transforms low-level linguistic information into a stable pedagogical contract.

2.4 Tense, Aspect, and Mood

Tense, aspect, and mood occupy an important place in ELA because they are among the areas that most often create difficulties for learners of English. In many learning systems, this kind of grammatical information is either hidden inside an opaque model or presented too superficially. It is also often taught through isolated examples and explanations that are detached from living language in context, which can create a sense of artificiality and a gap between the rule and its actual use. This, in turn, may lead to frustration and make it harder for the learner to move from memorizing a rule to genuinely understanding it.

ELA follows a more structured approach. The system first obtains the linguistic structure of a sentence through automatic analysis, then identifies grammatical features related to verb form, and only after that selects an appropriate pedagogical explanation. In some cases, this explanation is chosen from more direct ready-made formulations; in others, it is built from more general templates that are then adapted to the specific sentence. This preserves a clear connection between the actual grammatical form of the sentence and the way it is explained to the learner.

This design is important not only technically but also methodologically. It makes it

possible to test different stages of the analysis separately, to identify errors more precisely, and to avoid reducing grammatical interpretation to a single opaque generative step.

2.5 CEFR Estimation

CEFR is used in ELA as a practical way of describing the difficulty of language material in a form that is understandable both to learners and to teachers. Within the project, CEFR is not treated as an independent pedagogical theory or as a universal explanation of language acquisition. Its role is much more applied: it serves as an additional layer of interpretation that helps characterize the difficulty of sentences, phrases, and individual linguistic elements within the overall structure of analysis.

During development, several approaches to this task were explored, including heavier classifier-based models and lighter tabular strategies. Over time, the project moved toward an architecture in which CEFR inference is integrated into the overall contract-based pipeline as a separate and controlled stage. The system first constructs a structural representation of the sentence, then extracts features related to grammatical form, syntactic organization, and overall constructional complexity, and only after that predicts the CEFR level. This solution proved more suitable both for integration into the general RLE architecture and for the practical maintenance of the system, because it provides a more transparent and testable inference path than a fully opaque end-to-end approach.

This reflects a broader principle underlying ELA: wherever possible, the project prefers simpler and more auditable components when they better serve the goal of explainability and fit more naturally into a unified structure of analysis. In the case of CEFR, this means that what matters is not only the final predicted level, but also the fact that it appears as part of an interpretable processing chain rather than as an isolated black-box output.

2.6 Sequence-to-Sequence Models and T5

Sequence-to-sequence models occupy an important place in ELA because the task of pedagogical note generation is not limited to classification alone. After the system determines the grammatical structure of a sentence and selects an appropriate explanation, that explanation still has to be expressed in a form that is clear and natural for the learner. This is the stage at which T5-style models become useful, since they make it possible to transform a structured representation of the sentence into coherent pedagogical text. In the project, this line was not used as a standalone alternative to the rest of the architecture, but as a component within a broader pipeline in which generation depends on the already constructed RLE contract and on previously identified grammatical features.

At the same time, practical work on the system showed that the quality of the result depends not only on the model itself, but also on the form in which the training signal is presented to it. The most important improvements did not come simply from increasing model size or training for more epochs, but from moving away from raw and lexically tied formulations toward more stable structural templates. In the project, this took the form of canonicalized notes, placeholder-based templates, and a two-stage approach in which the appropriate type of note is selected first and its final wording is produced only afterwards. It was precisely these decisions that led to the later conclusion that the system generalizes better when it operates not only on words, but also on stable structural patterns. In this sense, the role of T5 in ELA is not free generation from scratch, but controlled linguistic realization of an already identified structural interpretation.

2.7 Automatic Speech Recognition and Translation

Work with authentic language material in ELA is not limited to text alone. A substantial part of real English exists in the form of audio and video content, where ordinary grammatical difficulties are accompanied by speech rate, pronunciation, spontaneous disfluencies, overlapping utterances, and the problem of aligning text accurately with the temporal structure of media. For this reason, the project was extended toward multimodal processing, including automatic speech recognition, sentence segmentation, subtitle generation, translation, and the subsequent integration of these results into the overall structure of analysis.

During the development of the project, this line gradually evolved from an experimental addition into an important subsystem in its own right. The codebase and documentation record a local path built primarily around Whisper for transcription and M2M100 for translation, while additional providers were considered as extensions or comparative alternatives. The practical value of this approach lies in the fact that the user can work not only with text, but also with real media content, receiving transcription, translation, subtitles, and further linguistic analysis within a single system. At the same time, this part of the project proved to be one of the most complex in terms of integration, because sentence boundaries, timestamps, translated fragments, and visual artefacts all have to remain synchronized across several layers of processing. In this sense, the multimodal direction makes ELA more than a text analysis system, turning it into a broader learning environment, while at the same time requiring a significantly higher level of engineering coordination between components.

2.8 Explainability as a Systems Property

One of the important ideas underlying ELA is that explainability in a language-learning system cannot be reduced to the final output of a model alone. The user should be able to see not only the result, but also the structure on the basis of which that result was produced. For this reason, explainability in ELA is treated as a property of the system as a whole: it depends on how the internal representation of data is organized, how validation stages are carried out, how results are linked to one another, and how they are ultimately presented to the user through the interface.

This is precisely why the intermediate RLE contract plays a central role in the project. Its function is not merely to store or transfer data between components, but to keep in a unified form the results of syntactic analysis, grammatical interpretation, translation, pedagogical notes, and visualization. In this way, explanation in ELA is built not as an isolated model response, but as a coordinated chain of related representations that can be inspected, validated, and shown to the learner.

2.9 Gap Analysis

The main gap addressed by ELA lies in the absence of a single system capable of working with authentic English material in different forms and transforming it into a structured and explainable representation suitable for learning, analysis, and further use. Existing solutions may offer separate functions such as translation, subtitles, grammar hints, or media playback, but they usually do not combine all of these capabilities into one coherent environment built around the linguistic structure of the material itself.

This is precisely where ELA differs. The project brings together:

- work with authentic text, audio, and video content;
- hierarchical representation of linguistic structure;
- explicit validation of the intermediate contract;
- learner-facing visualization and export of results;
- a research line concerned with the generation of pedagogical notes through structural abstraction.

In this way, ELA is not just another narrow tool for an isolated task, but an attempt to bring together within a single architecture several directions that in existing systems usually remain separated.

Chapter 3

Previous Research Foundation and Project Evolution

3.1 Relationship to the Earlier Research Thesis

This thesis is not the first formal study devoted to ELA. An earlier research thesis established the initial conceptual foundations of the project and defined it as a system intended to support bilingual comprehension of authentic English materials. That work formulated a key principle which remains central at the present stage as well: authentic language material should not be simplified or rewritten, but should instead become more accessible through structured and explainable analysis. It was in this context that the idea of Recursive Linguistic Elements was first introduced, along with the representation of a sentence as a hierarchy of interconnected linguistic units and the role of the interactive visualizer as a tool through which the user can explore the internal structure of the material.

At the same time, the earlier work was primarily research-oriented and conceptual in character. Its purpose was to define the problem space, justify the pedagogical and technological motivation of the project, establish the object model, and show the general direction in which such a system could develop. It already considered a hybrid architecture combining deterministic linguistic analysis with generative AI enrichment, and it presented a working desktop prototype with a basic visualization of results. However, its main emphasis still remained on formulating the initial research framework and demonstrating the fundamental feasibility of such an approach.

In the research part of the present project, this line was taken further. The original hypothesis about explainable structural analysis of authentic material was not only preserved, but developed into a more specific hypothesis: that the grammatical structure of language can be represented through stable recursive patterns suitable for machine learning and for subsequent pedagogical interpretation. Within the current work, this hypothesis was not only formulated but also investigated experimentally, including through

work on pedagogical notes, structural abstraction, placeholder-based representations, and the dependence of model quality on the form of the input representation and on the note inventory.

The present thesis continues this foundation, but shifts the center of gravity from conceptual justification to implementation, integration, and critical evaluation of the system as an engineering object. If in the earlier work ELA appeared primarily as a research idea and a prototypical direction, here it is treated as a developed architecture that includes a real processing pipeline, a contract-based data representation, multimodal components, a user interface, and experimental lines related to pedagogical notes, CEFR, and structural abstraction. For this reason, the main contribution of the current work should be seen not in restating the original principles, but in what was actually brought into a working state, integrated into a unified system, and tested in practice.

3.2 From Conceptual Prototype to Implemented System

The transition from the earlier research prototype to the current state of the project took place along several connected directions. First, the architectural logic of the system changed. At the earlier stage, the project relied more heavily on a conceptual design and on a looser integration of external AI services, whereas in the current implementation the central role is played by a more rigid and verifiable foundation: automatic linguistic analysis, contract-based data representation, interpretive rules, and consistent validation of intermediate results. The generative components did not disappear, but they came to be treated not as the source of structural truth, but as an additional enrichment layer within the system.

Second, the RLE contract itself underwent substantial development. In the earlier work, it primarily expressed the general idea of representing a sentence through the levels of sentence, phrase, and word. In the current project, it was transformed into a much stricter and more formalized structure. It now includes node identifiers, parent-child relations, source text spans, syntactic dependencies, service metadata, translations, linguistic notes, and additional enrichment layers. As a result, RLE became not simply a conceptual model, but a real contract that connects different parts of the system to one another.

Third, the project acquired a full operational layer. It now includes a runtime service, client persistence, media processing, frontend routes, export paths, and a much broader testing surface. All of this brought ELA closer to the form of a genuinely usable software system, but at the same time made the engineering limitations of the project more visible. At the conceptual level, many inconsistencies could remain in the background; in the

implemented system, such points begin to appear as actual problems of integration, synchronization, and maintenance.

3.3 Practical Evolution Through Repository Milestones

The practical evolution of ELA can be traced especially clearly through the repository history, because many of the important decisions in the project were first recorded in specifications, reports, and intermediate artefacts, and only then moved into code and infrastructure. This makes it possible to observe not only the final architecture, but also the process through which it was formed. The commit history shows that the project did not develop as a linear movement toward one larger model, but as a gradual strengthening of its structural foundation, a refinement of the contract, a redistribution of roles between components, and a steady expansion of the platform toward a multimodal and user-facing system.

One of the earliest stable directions was the strengthening of the deterministic core. In the repository this is visible in a series of changes related to the normalization of the notes pipeline, stricter frontend contract validation, the alignment of the strict contract with JSON Schema, and the steps connected with the development of the runtime notes pipeline and text analysis through an intermediate contract. At this stage, the project clearly fixed an important principle: before generative components could be trusted, the system had to possess a sufficiently reliable and verifiable structural baseline. For this reason, the earlier implementation work was focused not on free generation, but on the construction of a controlled contract, interpretive rules, and stable validation of intermediate results.

The next important step was the development of the RLE contract itself and of the pedagogical structure attached to it. The project history shows how the contract model gradually became stronger: stricter fields were introduced, the logic of note blueprints was stabilized, note context was unified, and more explicit classifier-oriented datasets and blueprint-driven pipelines were then formed. This is reflected both in the documentation and in commits related to the unification of the contract template note pipeline, template expansion strategy, full-ladder classifier datasets, and later note-id experiments. As a result, RLE ceased to be only a research idea and became a real working interface between sentence analysis, pedagogical notes, the CEFR layer, the runtime service, and the visualizer.

A separate line of development concerned the modelling strategy of the project. The commit history reveals an important shift: the role of T5 gradually became narrower and more controlled, while the task of selecting structure and pedagogical interpretation

moved more clearly toward a classifier-first design. This is particularly visible in the materials from late February and early March 2026, when the project introduced grammar curriculum classifier specifications, full-ladder CEFR pipelines, joint tabular baselines, controlled blueprint generation, and quality loops, while the generative model was increasingly treated not as the source of pedagogical structure itself, but as a tool for its controlled linguistic realization. In other words, the project arrived at a more defensible architecture in which deterministic parsing, rules, classifier outputs, and note blueprints form the core, while T5 operates within much stricter boundaries.

No less important was the move toward a client-first and PWA-oriented architecture. The commits make it clear how the project moved away from the state of a "pipeline plus prototype interface" toward a more complete application system: local SQLite persistence, client-side storage, runtime media flow, project-scoped processing paths, mobile and PWA fixes, offline-oriented behavior, dedicated visualizer improvements, and support for export workflows all appeared over time. This movement was accompanied not only by functional growth, but also by an increasing number of engineering compromises: the closer the system came to a real user-facing product, the more visible became questions of state synchronization, frontend-backend stability, cache invalidation, and resume logic.

Finally, multimodal integration formed another major axis of development. The commit history shows that work with audio and video passed through several iterations: ASR strategies changed, sentence boundaries and word-level timestamps were refined, subtitle alignment issues were corrected, translation moved more than once between client-side and server-side paths, and the media pipeline was gradually expanded with translation polling, render jobs, caching, and artefact generation. These changes make especially visible how the original idea of ELA as a tool for working with authentic language expanded from text analysis into a broader learner workflow involving files, media, subtitles, translation, phonetics, visualization, and export. For this reason, the current thesis must take into account not only the strengths of the architecture, but also the real regressions, integration mismatches, and runtime fragility that became visible precisely because the project developed into an end-to-end system rather than remaining at the level of a conceptual prototype.

3.4 Research Continuity

Despite the considerable expansion of the architecture and the increasing complexity of the system as a whole, the research continuity of the project remains clearly visible. Both the earlier thesis and the current work preserve the same underlying educational idea: authentic English material should not be replaced by simplified versions, and support for the learner should instead be built as a layer of structured interpretation over the original text, audio, or video. In the present project, however, this line was not only preserved

but also significantly strengthened through new experimental investigation.

In particular, the study conducted on the task of pedagogical note selection showed that a richer structural input representation in the form of an RLE contract produces better results than more superficial ways of representing the sentence. In addition, a series of experiments with increasing numbers of unique note types showed that the structural form of the notes themselves also affects model behaviour: raw notes achieved higher absolute accuracy in the tested range, whereas templated notes with placeholders demonstrated a more favourable dynamic as the note space became larger. In this way, the original research idea of ELA received not only conceptual support in the current work, but also partial empirical confirmation.

This is where the real continuity between the two phases of the project lies. If the earlier thesis mainly formulated the general principle of explainable structural support for authentic language, the current work translates that principle into a stricter computational form and begins to test it experimentally. As a result, the continuity of the project is visible not only in the preservation of its original educational motivation, but also in the development of the research hypothesis itself: namely, that the structural organization of language can serve as a stable and learnable basis for pedagogical interpretation.

The present results also make it possible to outline further directions for research. First, the observed dynamic needs to be tested on a substantially broader set of pedagogical notes, that is, with a much larger number of distinct note types available for the system to choose from. The scale of this note space remains the main limitation of the current experiment. Second, it remains a promising direction to investigate which forms of RLE representation best support note selection and subsequent generalization to new language examples. Finally, the current classifier-first architecture opens the way to a stricter separation between the selection of pedagogical interpretation and its later linguistic realization, which may form the basis of the next stage of ELA's development.

3.5 What Is New in This Thesis

The novelty of the present thesis lies not only in the fact that the concept of ELA was brought to the level of a working system, but also in the fact that the project led to a clearer formulation of the research idea itself. In the current work, language is treated as a structure that can be represented through Recursive Linguistic Elements, that is, through a recursive hierarchy of sentence, phrase, and word levels. Each such element is described not through specific words, but through abstract grammatical attributes, including node type, tense, aspect, mood, syntactic role, and CEFR-related characteristics. In this formulation, the model operates not simply on surface text, but on sequences of structural patterns, which makes it possible to generalize to new language examples that preserve the same grammatical shape.

An important part of the novelty of the thesis is that this direction was not only formulated theoretically, but also subjected to experimental testing. The experiments carried out in the project showed that a richer structural input representation in the form of an RLE contract produces better results in pedagogical note selection than more superficial forms of sentence representation. In addition, the series of experiments with increasing numbers of unique note types showed that the structural organization of the notes themselves also affects model behaviour and its capacity for generalization. In this way, the present thesis contributes novelty not only as an engineering implementation of ELA, but also as a partial empirical confirmation that the grammatical structure of language can serve as a stable and learnable basis for pedagogical interpretation.

3.6 Methodological Approach of This Thesis

The methodology of the present work was structured in the following way: first, a general approach was formulated for solving a particular project or research task, and then work within that approach proceeded iteratively. Each such cycle included the implementation of a component, pipeline, dataset, or experiment, the collection of feedback through tests, metrics, reports, and practical results, and then either the further improvement of the chosen solution, its partial revision, or its complete abandonment in favour of a new approach. In this way, the project did not develop according to a strictly linear scheme, but through recurring cycles of testing, correction, rollback, and reconstruction of solutions. It was in the course of this work that the research hypothesis itself emerged: namely, that the structural organization of language can serve as a stable and learnable basis for pedagogical interpretation.

This methodology was applied both to the engineering side of ELA and to the development of its research line. It was used in work on the RLE contract, dataset construction, the choice between generative and classifier-based approaches, CEFR inference, pedagogical notes, and the multimodal runtime pipeline. In some cases this process led to the gradual strengthening of the original solution, while in others it led to a change of architectural direction. For this reason, both the final form of the ELA system and the research conclusions of the thesis emerged not as the execution of a fully fixed plan, but as the result of sequential iterative work in which each next version was refined under the influence of the feedback received.

3.7 Project Risks

For the present thesis, the main risks are not only ordinary engineering risks, but also risks connected with the interpretation of the project's results. Because ELA developed iteratively, and because its architecture, datasets, and model components changed over

the course of the work, there is a risk of drawing conclusions that are too broad from intermediate states of the system. This applies both to the general description of ELA's capabilities and to the interpretation of particular experiments and metrics.

In the work itself, such risks appeared in quite concrete ways. For example, at the earlier stages of the modelling line, the role of T5 was understood more broadly than in the final architecture of the project. Over time, the system moved toward a stricter separation of tasks: the selection of a pedagogical note came to belong primarily to the classification stage, while the generative model came to be used in a more limited way, mainly for the linguistic wording of an interpretation that had already been selected. Likewise, quantitative results cannot be interpreted outside the conditions in which they were obtained. Comparisons between direct text-based notes and templated notes with placeholders, between more superficial input and input in the form of an RLE contract, and between experiments with different numbers of unique note types all produced different kinds of model behaviour. This means that headline numerical results from different stages of the work cannot be compared without taking into account the specific experimental setup.

Integration risks also appeared in observable practical form. In different parts of the project, repeated corrections were required in how the system split audio into sentences, how it synchronized subtitle text with the time structure of media, how it stored and reused already processed results, and how it coordinated state between the user interface, the runtime layer, and the multimodal pipeline. Finally, the scale of ELA itself creates a separate risk of uneven maturity: the system simultaneously includes structural linguistic analysis, CEFR, pedagogical notes, classifier-based and generative components, audio and video processing, visualization, and export. For this reason, by the time the work is completed, different subsystems are inevitably at different degrees of stabilization.

Recognizing these risks matters not as a formal disclaimer, but as part of the research honesty of the thesis. For this reason, it is necessary to distinguish clearly between three groups of results.

Already implemented and substantially stabilized:

- the RLE contract as the main structural representation of the sentence;
- the decomposition of material into sentence, phrase, and word levels;
- the basic linguistic analysis combining parser-based processing and rule-based grammatical interpretation;
- the contract-oriented runtime approach;
- CEFR as an integrated layer of difficulty interpretation;
- the general user-facing visualization layer;

- the basic multimodal processing of text, audio, and video.

Still at the experimental stage:

- the exact form of the input representation for pedagogical note selection;
- the comparative role of direct text-based notes and templated notes with placeholders;
- the classifier-first scheme for note selection;
- the controlled use of T5 after classification;
- the question of which forms of structural abstraction best support generalization.

Best treated as directions for further development:

- the expansion of the space of pedagogical notes;
- the construction of larger and cleaner datasets;
- the further separation between the selection of interpretation and its later linguistic realization;
- broader experimental testing of the hypothesis about structural patterns;
- user studies of the pedagogical effectiveness of ELA.

Chapter 4

The RLE Data Model

4.1 Design Rationale

The RLE contract forms the structural foundation of the entire ELA system. Its purpose is to represent an English sentence not as a flat string of text, but as a recursive hierarchy of interconnected element-nodes organized at the levels of sentence, phrase, and word. Each such node stores not only its textual content, but also a set of abstract linguistic characteristics, including node type, syntactic role, morphological and grammatical features, links to other nodes, and additional layers of interpretation such as CEFR, translation, pedagogical notes, and other enrichments.

It is precisely this structure that makes RLE both a working contract for the internal components of the system and a convenient form of representation for further processing. It makes it possible to connect parser-based analysis, rule-based interpretation, classifier and generation components, runtime services, and user-facing visualization within one coherent model. In this way, RLE is needed not only for data storage, but also to ensure that the same structural form can be used for analysis, validation, machine processing, visual rendering, and pedagogical interpretation.

4.2 Hierarchy

The hierarchy of the current RLE contract is organized as a recursive tree in which `Sentence` always serves as the root of the entire structure. This root node defines the boundaries of the sentence and brings together the general information about the utterance: its textual content, core grammatical characteristics, translation, CEFR level, pedagogical notes, service identifiers, and the array of child elements stored in `linguistic_elements`. In this way, the sentence functions not simply as a string of text, but as the top-level node from which the whole structure of analysis unfolds.

Below the root, the tree is built flexibly. Within a sentence, there may be both `Phrase`

nodes and `Word` nodes, depending on how the analysis of a particular fragment has been constructed. `Phrase` nodes are used to represent intermediate branches of the tree, that is, syntactically and semantically meaningful groups within the sentence. In the actual contract, these may include such nodes as `verb phrase`, `prepositional phrase`, `noun phrase`, `relative clause`, and other phrasal structures. These branches are needed in order to connect the sentence as a whole with its internal parts and to store at these intermediate levels their own roles, spans, translations, pedagogical notes, and other fields whenever such information has been assigned to the node.

At the same time, phrasal nodes do not form one fixed layer. They may directly contain child words, but they may also break down into embedded subphrases when the structure of the sentence requires a deeper branching organization. For this reason, the hierarchy in RLE is not a rigid ladder of predefined stages, but a recursive tree with potentially nested branches.

`Word` nodes represent individual tokens and function as the leaves of the tree, that is, as terminal elements that are not further divided inside the contract. In the current code, such nodes carry fields related to the token text, its position in the sentence, its part of speech, grammatical role, dependency links, `head_id`, `parent_id`, `source_span`, and, where available, translation, phonetics, CEFR assessment, grammar classes, note blueprints, and other enrichments. A leaf node may therefore be attached either directly to the sentence root or to one of the intermediate phrasal nodes, depending on how the structure was built.

The basic logic of the RLE hierarchy can therefore be summarized as follows: `Sentence` always forms the root, `Phrase` nodes form intermediate branches, and `Word` nodes most often function as the leaves of the tree. It is this recursive organization that makes the contract suitable for analysis, validation, visualization, machine processing, and pedagogical interpretation within one and the same data model.

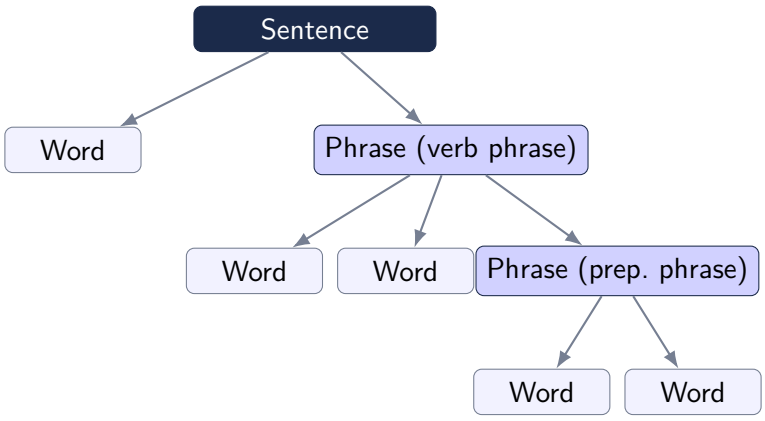

Figure 4.1: Illustrative RLE tree: a `Sentence` root with a direct `Word` child (uncovered token) and a `Phrase (verb phrase)` that itself contains two `Word` nodes and a nested `Phrase (prep. phrase)`.

4.3 Core Fields

In its current form, the RLE contract uses not just a set of isolated fields, but several functional groups of fields, each responsible for its own layer of representation. The basic layer defines the identity of the node itself. It includes `type`, which determines whether the node is a sentence, phrase, or word, `content`, which stores the textual content of the node, and `part_of_speech`, which records its grammatical category in a form suitable for further interpretation. These fields give the node its minimal form as an independent element of the tree.

The next important group of fields is connected with the grammatical state of the node. In the current contract, this includes `tense`, `aspect`, `mood`, `voice`, `finiteness`, and `tam_construction`. These fields make it possible to store not only the surface text form, but also the structural grammatical interpretation of the node, which is especially important for verbal and predicative constructions. At the same time, the actual data show that such fields are used not only at the level of the whole sentence, but also at the level of phrase and word nodes whenever a corresponding grammatical characteristic has been assigned to that element.

A separate layer is formed by the fields that support structural navigation and tree connectivity. These include `node_id`, `parent_id`, and `linguistic_elements`, which define the place of the node inside the overall recursive structure, as well as `source_span`, which makes it possible to align the node with its exact range in the source text. For word- and phrase-level nodes, `grammatical_role`, `dep_label`, and `head_id` are additionally important, because they connect the contract not only to the visible hierarchy of the tree, but also to syntactic relations inherited or derived from parser-based analysis.

A substantial part of the current contract consists of fields for interpretation and pedagogical enrichment. These include `linguistic_notes`, `grammar_classes`, `note_blueprints`, and `note_generator_version`. Through them, the node is linked to pedagogical notes, to the classification of the grammatical phenomenon, and to the explanatory form that can later be shown to the learner or refined in a controlled generation pipeline. An important feature of the current architecture is that these fields are no longer treated as incidental textual additions, but as part of a coordinated contract-safe approach in which structural truth, classifier outputs, and later rendering are intentionally separated.

Another group of fields supports the multimodal and learner-facing enrichment of the node. In the current contract, this includes above all `translations`, `active_translation_provider`, `phonetic`, `synonyms`, and `cefr_level`. These fields make the contract suitable not only for internal analysis, but also for real learner-facing use, because it is through them that translation, phonetic information, vocabulary support, and difficulty assessment are embedded into the node structure.

Finally, the contract contains service and control fields needed for validation and for

the stable functioning of the system. These include `schema_version`, as well as trace-related and quality-related fields that are used by the validator and runtime to check structural correctness, consistency of nesting, and the quality of individual stages of processing. The presence of such service fields shows that the current RLE contract is not merely a format for storing linguistic annotation, but a working interface between analysis, enrichment, validation, generation, and user-facing presentation of the result.

4.4 Validation Modes

In the current implementation of the contract, two validation modes are used: `v1` and `v2_strict`. They differ not only in degree of strictness, but also in the stage of system development that they effectively reflect. The `v1` mode preserves compatibility with earlier variants of the contract and allows more permissive forms of data representation. In particular, TAM fields are validated there as string values, and some structural fields that became mandatory in the later architecture may still be absent. This mode is needed primarily for working with the legacy side of the project, with intermediate artefacts, and with those parts of the pipeline that originated before the current strict contract was finalized.

By contrast, `v2_strict` reflects the current target state of the production-facing contract. In this mode, each node is subject to additional mandatory requirements for `node_id`, `source_span`, `grammatical_role`, and `schema_version`. Moreover, in strict mode `schema_version` must be explicitly equal to `v2`. An important difference also concerns nullable fields: for `tense`, `aspect`, `mood`, `voice`, `finiteness`, and `features`, strict validation requires either a string value or a real JSON `null`, but not the string marker `"null"`. This rule is especially important in those parts of the system where the contract passes through runtime, inference, and frontend payloads, because at that point what is required is not only formal validity, but a uniform interpretation of the data across all layers of the system.

In `v2_strict`, the logic of validating node content is also strengthened. For example, for `Sentence` and `Phrase` nodes, `linguistic_notes` must not only be present, but in strict mode must contain non-empty content. If notes are represented as a three-level object (`elementary`, `intermediate`, `advanced`), then at least `intermediate` must be non-empty. If they are represented as a list of strings, then the list must contain at least one non-empty note. In a similar way, strict mode checks the consistency of `parent_id`, the uniqueness of `node_id`, the correctness of `head_id`, the admissibility of child node types, and the rule that a `Word` node cannot itself contain child elements. For some grammatical configurations, special constraints are also added, for example the `modal_perfect` policy, in which the combination of `mood`, `aspect`, `tense`, and `tam_construction` must be kept in strict agreement.

The difference between `v1` and `v2_strict` in the current project therefore does not consist in an abstract contrast between a "soft" and a "strict" check, but in the transition from a more tolerant compatibility mode to a mode that serves the mature form of the contract. At the same time, the code itself shows that full compatibility between these two modes is still not perfectly smooth. The most sensitive points are connected with the handling of `null` values in TAM fields, with the normalization of the shape of `linguistic_notes`, and with the fact that runtime and frontend now expect a stricter and more uniform structure than was acceptable at earlier stages of the project.

4.5 How the Contract Evolved in Practice

The RLE contract did not appear at once in the form in which it is used in the current version of ELA. Its development was gradual and reflected the broader evolution of the project from a conceptual prototype to an implemented system. Many parts of this process have already been discussed separately in earlier sections: the development of node hierarchy, the strengthening of validation, the growing role of node identity, and the integration of grammatical interpretation, pedagogical notes, CEFR, translation, and the runtime layer. In the present section, it is important to bring these lines together, because only then does it become clear how the contract gradually turned from a supporting format into the central working interface of the whole system.

From the beginning, the contract was conceived as a recursive structure in which a sentence is represented as a tree of interconnected element-nodes. At the earlier stages, however, the main task was above all to obtain a workable form of this representation and make it usable in practice. As the project moved from general concept to real pipeline, the contract gradually took on more and more functions: it had to store not only the sentence tree itself, but also syntactic relations, grammatical interpretation, translation, pedagogical notes, CEFR, user-facing visualization, and the operation of the runtime layer.

For this reason, the evolution of the contract did not take the form of an abstract expansion of the schema, but rather of a response to concrete requirements coming from other parts of the project. The appearance of `node_id` and `parent_id` was tied to the need to identify nodes reliably within the tree and to connect them to one another in the interface and in the internal logic of processing. The field `source_span` became necessary for the exact alignment of a node with the source text, and therefore also for alignment, targeted editing, translation, and multimodal scenarios. The fields `grammatical_role`, `dep_label`, and `head_id` appeared because the contract had to become not merely a nested display format, but a linguistically meaningful structure connected with parser-based analysis. In the same way, trace fields, quality-related fields, and a stricter validation scheme became necessary once the system was no longer only constructing the

contract, but also checking the quality of intermediate and final results.

It is especially important that the recursive nature of the RLE contract was not a later addition, but belonged to the foundation of the model from the start. As the project developed, however, this recursive structure came to be used in a progressively stricter and more consistent way by different parts of the system. At earlier stages, the main goal was to establish the idea itself and to obtain a working representation of the sentence as a tree of element-nodes. Later, it became necessary for the visualizer, runtime payloads, validator logic, note-generation pipeline, and other components to rely on the same structural logic without divergence. This made the contract more coherent, but at the same time more demanding: once node identity, field ownership, nesting rules, and validation modes became central, problems of schema drift, differences between legacy and strict modes, and the handling of `null` values in grammatical fields turned from local technical details into visible architectural questions.

In this way, the practical evolution of the RLE contract reflects the general logic of ELA's development as a whole. At first, the contract was needed as a way of fixing the result of analysis; later, it became a means of coordinating different components of the system; and in its current state, it has become the central working interface through which analysis, validation, enrichment, generation, and user-facing presentation of the result all pass. In this sense, the history of the RLE contract is not a separate technical line, but one of the main axes along which the entire project developed.

4.6 Contract Example

Listing 4.1 presents a shortened but current example of a sentence-level RLE contract node. It does not show every possible field, but only the main groups of data that in the current system connect the structural analysis of the sentence, grammatical interpretation, pedagogical notes, and learner-facing enrichments.

Listing 4.1: Current sentence-level RLE contract example

```
{
  "type": "Sentence",                          // node type: root sentence node
  "content": "She came to him towards morning.", // textual content of the
    node
  "tense": "past",                            // tense
  "aspect": "simple",                         // aspect
  "mood": "indicative",                       // mood
  "voice": "active",                          // voice
  "finiteness": "finite",                     // finiteness
  "tam_construction": "past_simple",          // generalized TAM construction
```

```
"linguistic_notes": [                          // learner-facing pedagogical
 note
  "Explain past simple verb form and completed past-time meaning."
],

"schema_version": "v2",                      // contract version
"part_of_speech": "sentence",                // grammatical category of the
 node
"node_id": "n1",                             // unique node identifier
"parent_id": null,                             // root node has no parent
"source_span": {                               // exact span in the source
 text
  "start": 0,
  "end": 31
},
"grammatical_role": "clause",                // grammatical role of the node

"translations": {                              // translations attached to the
   node
  "m2m100": {
    "source_lang": "en",
    "target_lang": "ru",
    "text": "Она пришла к нему под утро."  // EN->RU (M2M100)
  }
},
"active_translation_provider": "m2m100",  // currently active translation
 provider

"phonetic": {                                // IPA transcription (espeak-ng)
  "uk": "Si keIm tu hIm tu'wO:dz 'mO:nIN",
  "us": "Si keIm tu hIm tu'wO:rdz 'mO:rnIN"
},

"synonyms": [],                                // lexical enrichment
"cefr_level": "A2",                          // CEFR difficulty estimate

"grammar_classes": [                           // grammatical classification
 of the node
  {
    "class_id": "past_simple_affirmative",
    "confidence": 0.9
```

```
    }
  ],

  "note_blueprints": {                          // classifier-owned note
    blueprints
    "elementary_text": "Identify the past simple statement and the finished
    past action.",
    "intermediate_text": "Explain past simple verb form and completed past-
    time meaning.",
    "advanced_text": "Describe how the past simple places the event in a
    finished narrative timeline."
  },

  "note_generator_version": "controlled_t5::blueprint_rewrite", // note-
    generation path version

  "linguistic_elements": [                      // child nodes in the recursive
     tree
    { "...": "Phrase / Word child nodes omitted here for brevity" }
  ]
}
```

4.7 What the Canonical Sample Shows

The canonical sample contract matters not only as an illustration of structure, but also as a compressed reflection of the current architectural decisions of the project. It shows that a sentence-level node is no longer a simple container for text. It simultaneously carries structural fields, grammatical interpretation, translation, CEFR assessment, pedagogical notes, note blueprints, and links to the child elements of the tree. In other words, in the current state of the system, one and the same node acts as the meeting point of several layers at once: parser-based structure, rule-based grammatical interpretation, predictive enrichment, and learner-facing presentation.

From a practical point of view, this sample makes several important features of the current contract visible. First, translation, phonetics, CEFR, and pedagogical fields are embedded directly into the node rather than being loaded as external auxiliary objects. Second, sentence-level and phrase-level nodes can genuinely carry their own TAM characteristics, grammatical roles, and learner-facing notes, rather than functioning merely as intermediate containers. Third, the fields `node_id`, `parent_id`, `source_span`, `dep_label`, and `head_id` make the node not simply a part of a visible tree, but a stable

unit for navigation, validation, editing, and coordination across the internal layers of the system.

This example is especially important in the context of field ownership. The document `contract_field_ownership_2026-03-04` fixes the intended architectural boundary quite clearly: the shape of the tree, node identity, syntactic relations, and the basic grammatical interpretation should remain under the control of parser-based and rules-based components; `cefr_level` should be determined through a separate predictive layer; and the generative model should not invent structural truth, but should only formulate already selected pedagogical interpretation in learner-facing wording. For this reason, the fields `grammar_classes`, `note_blueprints`, and `linguistic_notes` cannot be treated as the same thing: in the current architecture they belong to different stages of processing and to different sources of truth.

At the same time, the current state of the code shows that this architectural boundary has largely been established, but has not yet been made perfectly clean everywhere. On the one hand, runtime and validator already treat the contract as a coherent structure, normalize the shape of `linguistic_notes`, and enforce strict behaviour in `v2_strict`. On the other hand, the inference path still contains a tension between blueprint-owned and generation-owned fields: part of the code already uses `note_blueprints` as the source for user-facing notes, but the controlled T5 rewrite still modifies the blueprint values themselves. For this reason, the canonical sample is important not only as an example of the mature form of the contract, but also as a point through which it becomes visible which ownership decisions in the project have already been fixed, and which are still not fully cleaned up at the level of implementation.

In this way, the significance of the canonical sample does not lie in showing "all fields at once", but in making visible the current logic of the system as a whole: the contract functions not as a passive storage schema, but as an active boundary between structural truth, interpretation, prediction, generation, and learner-facing presentation of the result.

4.8 Why Hierarchical JSON

Hierarchical JSON was chosen in ELA not simply as a convenient technical format, but as a form of representation that corresponds to the nature of linguistic structure itself. In the project, a sentence is treated not as a linear sequence of words, but as a hierarchy of grammatical and lexico-grammatical constructions expressed through the nesting of element-nodes. For this reason, the RLE contract is built as a recursive tree: this form more accurately reflects how language is actually organized across the levels of sentence, phrase, and word.

At the same time, this form of organization is important not only for theoretical reasons. It also allows the system to work with the same structure throughout all stages

of processing, from parser-based analysis and rule-based interpretation to pedagogical notes, runtime processing, visualization, and export. If the project had used a flat annotation scheme, the relations between levels would have had to be constantly reconstructed through external fields, additional assembly logic, and more complex synchronization between analysis, validation, and user-facing presentation of the result.

From a practical point of view, this format also proved more convenient for several tasks at once. It naturally matches the logic of the visualizer, where the tree does not have to be rebuilt from disconnected records. It makes it possible to use the same schema family for sentence-, phrase-, and word-level nodes. It is better suited to node-by-node validation and to rules that check not only individual fields, but also relations inside subtrees. Finally, it simplifies runtime storage, export, testing, and data exchange between system components, because the whole sentence is passed as a single structured object.

Table 4.1: Hierarchical RLE versus flatter annotation approaches

Criterion	Hierarchical RLE	Flat annotation
Structural nesting	Nesting is expressed directly inside the tree	Nesting must be reconstructed through external links
Interface representation	The visualizer can work with the tree directly	An additional tree-assembly layer is required
Validation	Supports checks of both individual nodes and subtrees	Easier for individual records, weaker for whole-structure validation
Pedagogical readability	Supports gradual disclosure of structure well	More likely to fragment the pedagogical view
Export and serialization	Well suited to JSON-based runtime, export, and testing workflows	More convenient mainly for tabular and spreadsheet-like formats

Chapter 5

Context–Pattern Duality and Placeholder Generalisation

5.1 Observed Problem

During the development of ELA, one recurring problem became increasingly visible: the system performed worse when learning depended too heavily on specific words and on fixed textual formulations. In such cases, it was more likely to memorize individual examples than to learn how to recognize the same construction in new sentences. This is especially important for the present project because ELA is expected to work not only with phrases it has already seen, but also with new sentences in which the words may change while the grammatical construction remains the same.

In practice, this problem became especially clear in the task of pedagogical notes. When an explanation was tied too closely to the literal form of a particular example, the system found it harder to apply that explanation to other sentences of the same type. In other words, the difficulty was that learning too easily attached itself to the surface form of the text and did not retain the structure of the construction strongly enough. This was the initial problem from which the later line of work on structural abstraction in ELA developed.

5.2 Observed Improvement

During the experiments carried out in the project, a fairly stable pattern became visible: the quality of the system improved when the representation of the data depended less on the literal surface form of the text and preserved more of the structure of the sentence or the note. This appeared not in one isolated test, but across several experimental lines in which both the models and the form of the input and target space were changed.

One of the clearest results was obtained in the task of pedagogical note selection. In

this experiment, two input variants for the classifier were compared: a more superficial representation of the sentence and a richer input in the form of an RLE contract. After switching to contract-based input, the proportion of examples in which the system placed the correct note in first position rose from ‘0.3519‘ to ‘0.3928‘. Even more importantly, the proportion of examples in which the correct note appeared at least among the three best candidates increased substantially, rising from ‘0.6037‘ to ‘0.7700‘. In practical terms, this means that a more structured representation of the sentence made the selection of pedagogical interpretation noticeably more stable even without moving to a more complex model.

An even stronger result was obtained in the CEFR line. In experiments where the difficulty level was determined not from raw text as such, but from structured features extracted from the contract representation of the sentence, classification quality became exceptionally high. In one tabular CEFR run, test-set accuracy reached about ‘99.6%‘, and in a later joint-profile experiment the test-set accuracy reached ‘100%‘. At the same time, comparison with the DeBERTa baseline on the same full-ladder evaluation showed a sharp quality gap: the neural baseline remained at about ‘16.1%‘ accuracy, whereas the tabular classifier, relying on already extracted structural features, recovered the correct CEFR labels almost perfectly. This does not by itself prove that any use of the contract will automatically guarantee such a result, but it shows very clearly that in the current project the structured representation of the sentence carries an exceptionally strong signal for difficulty classification.

Another important result appeared in the series of experiments that tracked classifier behaviour as the number of unique pedagogical note types increased. Here two regimes were compared: direct text-based notes and templated notes with placeholders. For direct notes, the system showed higher first-choice accuracy at a small and medium number of classes. For example, with ‘58‘ unique note types, the correct note was placed in first position in about ‘64.7%‘ of cases, whereas for templated notes this figure was much lower. However, as the number of unique note types increased, direct text-based notes lost quality more quickly: when the target space expanded from ‘58‘ to ‘97‘ classes, first-choice accuracy declined noticeably. Templated notes, by contrast, started from a weaker level, but their dynamic as the number of classes grew was more stable. This matters because it shows that the organization of the target space itself affects not only current accuracy, but also how the system behaves as the task becomes harder.

Finally, the negative result of the early template-ID experiment dated ‘2026-02-13‘ is also important. There the project encountered the opposite extreme: after balancing, the dataset became too heavily compressed, leaving only ‘511‘ rows and ‘10‘ unique targets, and both tested model lines fell completely into fallback behaviour in the control check. Across all ‘46‘ checked nodes, the final notes were taken not from the model output, but from the fallback path. This experiment showed that abstraction by itself does not

yet guarantee improvement. The gain appears not when the target space is mechanically compressed, but when a more abstract representation actually helps preserve transferable structure without destroying meaningful differences between alternatives.

5.3 Empirical Path to the Claim

At an early stage, the project was dealing with a fairly local engineering problem: the system had to produce pedagogically useful interpretations of sentences without reducing everything to hand-written rules and without losing control over the output. A natural first move was therefore to simplify the target space, make it more compact, and in that way make the modelling task easier. It was in this context that the early template-ID experiment of ‘2026-02-13‘ was carried out.

That step, however, produced an unexpected result. Formally, the system became more controlled, but no practically useful outcome followed. After balancing, the dataset shrank to only ‘511‘ rows and ‘10‘ unique targets, and in the small regression check both tested model lines fell completely into fallback behaviour. In other words, the first attempt to make the task more abstract turned out to be too crude: instead of isolating stable structure, the project produced a target space that had become so compressed that the model could no longer distinguish enough within it. This was an important moment because it showed that simply reducing variation does not yet produce useful generalization.

The next steps moved in a different direction. Instead of continuing to compress the target space mechanically, the project began to work more carefully with the representation of the sentence itself. Gradually, the RLE contract moved to the center. In this representation, a sentence was no longer treated as a flat string of words, but as a hierarchy of element-nodes carrying grammatical and interpretive attributes. This changed not only the storage format, but also the character of the learning task itself: the system could now rely not only on the text as such, but also on the internal organization of the construction.

After that, results from different parts of the project began to converge into one line. In the task of pedagogical note selection, richer input in the form of an RLE contract outperformed a more superficial representation of the sentence. In the CEFR line, structured features extracted from the contract representation produced almost perfect classification on the corresponding dataset. In the series of experiments with raw and templated notes, it became clear that even the organization of the notes themselves affected model behaviour as the number of classes increased: direct text-based notes were stronger in absolute accuracy in the tested range, but templated notes behaved more steadily as the target space became more difficult. Taken separately, these results belonged to different tasks, but together they pointed in the same direction: the system performed better when

it had access not merely to examples, but to the more stable structural relations behind the examples.

At the same time, an architectural shift was also taking place. The earlier logic of the project had still allowed for the possibility that one generative model could directly produce the whole pedagogical interpretation. But the repository history shows how this idea gradually gave way to a different organization of the system. The contract, grammar classes, CEFR, and note blueprints were increasingly fixed as separate layers with their own sources of truth, while the generative component was restricted to the linguistic realization of content that had already been selected. In this respect, the phase-1 execution report dated ‘2026-02-25‘ is especially important, because it records a working controlled path in which classifier-owned blueprint fields are preserved and generation adds only learner-facing wording.

If these steps are viewed together, they lead not merely to a local technical observation, but to a more general research conclusion. At first, the project was confronted with many concrete sentences, notes, and variants of their realization. It then became clear that neither simply accumulating these variants nor merely compressing them solved the problem. A workable result appeared only when the system began to isolate more stable patterns within this variety and rely on them for later classification, interpretation, and controlled generation. In other words, the path of the project gradually showed that useful generalization arises not from enumerating surface cases, but from discovering the more abstract structure that organizes those cases.

5.4 Interpretive Framework

The results obtained in the project can be interpreted through the following framework. During training, the system is exposed to a large number of concrete examples, and it is from this variety that more stable patterns are gradually extracted. During inference, the process in a sense runs in the opposite direction: the patterns that have already been formed begin to serve as the basis for interpreting new cases. In other words, during training many contexts lead to the formation of pattern, whereas during application of the model it is the pattern that helps recognize and organize new context.

An important point is that this scheme is not proposed as a strict formal theory. In the present thesis, it is used as an empirical explanatory framework that emerged from the path of the project itself. It provides the clearest way to describe what the experiments showed: improvement appeared when the system depended less on individual surface cases and relied more on the more stable structural organization of the data. In this sense, context–pattern duality is not offered as an independent proof, but as a way of bringing together into one interpretation the observations obtained in note-selection, CEFR, and placeholder-based processing experiments.

Within this framework, context means the observable concrete instances with which the system is confronted: individual sentences, note formulations, CEFR-labelled examples, and other realized cases. Pattern means the more stable abstract structure that can be extracted from such cases: recurrent grammatical organization, note classes, blueprint-like templates, and other regularities that remain visible when the lexical material changes.

The value of the framework lies in distinguishing these two levels without separating them completely. A pattern does not exist outside contexts: it is derived from many contexts during training. But once formed, it becomes a compact structural guide for interpreting new contexts during inference. This is why the framework is useful for the present thesis: it explains how the project moved from individual examples toward reusable structural representations without treating those representations as something detached from the data from which they emerged.

Figure 5.1: Context–pattern duality: training extracts stable patterns from many contexts; inference applies those frozen patterns to new unseen input.

5.5 Lexical Invariance

By lexical invariance, the present work means the following property: a pedagogically meaningful interpretation of a construction remains applicable even when the specific words in the sentence change. In other words, if two sentences differ in lexical content but are built according to the same grammatical or lexico-grammatical pattern, the system should, as far as possible, recognize the same structural basis in both and associate it with the same or a closely related pedagogical interpretation. In this sense, invariance here does not refer to the literal form of the text, but to the stability of structural meaning under changing verbal material.

This understanding follows directly from the results of the previous sections. Section ‘5.1‘ showed the initial problem: too much dependence on specific words made generalization weaker. Section ‘5.2‘ showed that quality improved when the system relied on a more structured representation of both input and target. Section ‘5.3‘ traced this line as the empirical path of the project, and Section ‘5.4‘ interpreted it through the context–pattern framework. For this reason, lexical invariance in the present section should not be understood as a separate new hypothesis, but as a more specific formulation of the same

underlying regularity: useful generalization emerges when the system learns to preserve structure while allowing lexical variation.

The practical meaning of the placeholder strategy lies exactly here: pedagogical interpretation becomes less dependent on specific words and more strongly tied to the construction itself. If the system learns to recognize and use the structural features of a sentence, then the same interpretation can be transferred to new examples even when their lexical content changes. This is especially important for ELA, because in language learning the meaning of a grammatical construction usually remains the same even when the particular sentence, nouns, or verbs are different.

The project artifacts support a limited but meaningful version of this conclusion. In later examples from the controlled note-generation pipeline, the system already shows exact sentence-level matches for several different types of constructions. At the same time, the results remain imperfect: phrase-level examples still reveal cases in which placeholder-based rendering becomes too literal and outputs technical labels instead of natural learner-facing wording. This matters because it clarifies the result itself. The project did not show that any abstraction automatically produces a good explanation. It showed something narrower but more reliable: transferability of interpretation becomes genuinely useful when the right structural information is abstracted and the final wording remains under tight control.

5.6 Scope of the Claim

The conclusion drawn in this work should remain bounded and carefully formulated. The present thesis does not claim that the ELA case reveals a universal law for all learning systems, and it certainly does not attempt to turn observed model behaviour into a result of physical scope. The contribution is narrower and at the same time more reliable: it concerns a specific class of tasks, a specific path of development, and a specific set of experimental observations within this project.

At the same time, within this more limited scope, the work leads to a fairly clear interpretation. During training, the system is in a conditionally open phase: it encounters many observable concrete cases from which more stable abstract patterns and sequences of patterns are gradually extracted. During inference, by contrast, the system acts in a conditionally closed phase: the patterns and sequences of patterns that have already been formed begin to be used for the interpretation of new examples and for the production of new meaningful output.

A useful analogy here is the process by which a human being learns a language. For a person who does not yet know the language, the linguistic environment also first appears as an open system: the learner encounters many separate words, constructions, and utterances, gradually identifies recurring patterns within them, and internalizes more

stable principles of organization. Later, once these patterns have been learned, the learner begins to use them in the opposite direction: not only to recognize the speech of others, but also to produce utterances of their own in the language being learned. In this sense, training can be understood as an open phase of extracting structure from complex variety, while inference can be understood as a closed phase of applying the structure that has already been extracted.

It is in this narrower sense that the contribution of the work should be understood. First, the project empirically observed that generalization improves when the system moves from more literal textual forms to more structured and abstract representations. Second, the context–pattern framework provides a coherent explanation of this dynamic by showing how more stable patterns are gradually extracted from many concrete cases and are then used to interpret new data. Third, this interpretation has not only descriptive but also practical value, because it helps in designing pedagogical generation tasks in ELA and in other systems where structural interpretation must be transferred to new data.

5.7 Cross-Domain Evidence: The Same Pattern in Weather Forecasting

The scope statement in the previous section deliberately keeps the conclusion within the bounds of ELA and pedagogical generation. However, during the same research period a similar structural observation appeared in another project belonging to a completely different domain. This matters not because the second domain automatically proves the first, but because the convergence arises independently, with different data, a different task, and a different form of signal. In this way, the observation made in ELA gains not universal proof, but a stronger cross-domain support.

In the companion weather forecasting project [15], the model was likewise not trained directly on the raw stream of observations as such. Instead, the multi-year time series for temperature, pressure, and wind speed were first decomposed into more compact analytical segments. Each segment was described not by individual values, but by a behaviour type and its parameters: for temperature, six ODE-derived equation types were used, and for pressure and wind, three. Across six years of observations, this produced ‘10,055‘ segments with a mean duration of ‘5.3‘ hours. After that, a transformer was trained not on raw measurements, but on sequences of such analytical descriptors, predicting the next regime rather than individual point values. On a one-week evaluation period in February 2026, which included a sharp pressure drop of ‘22‘ hPa, the system achieved a pressure MAE of ‘0.89‘ hPa at the 12-hour horizon, which was ‘34%‘ better than a naive persistence baseline.

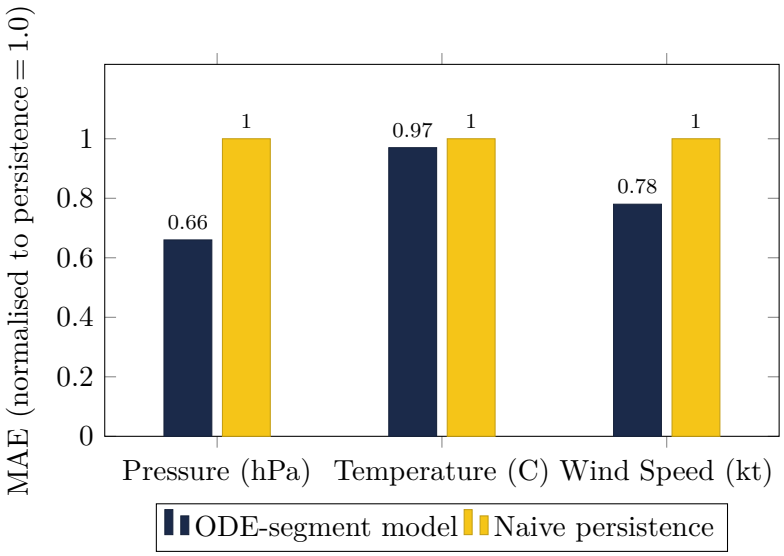

Figure 5.2: Normalised MAE for the ODE-segment forecasting model versus naive persistence across three variables (February 2026 evaluation). Values below 1.0 indicate improvement over baseline.

The structural parallel with ELA is fundamental. In the language project, the sentence is decomposed into Recursive Linguistic Elements and then represented through abstract node attributes and their relations. In the weather project, the flow of observations is first compressed into sequences of more stable analytical regimes. In both cases, the model is trained not on the surface of the signal itself, but on more compact structural units extracted from that signal. And in both cases, the practical gain appears when these units and their sequences become the main object of learning.

For this reason, the significance of the comparison goes beyond a local technical analogy. In both projects, despite the obvious differences between language and meteorology, the same research logic becomes visible. First, a complex system is observed as a stream of many concrete realizations. Then, more stable abstract patterns are extracted from that stream. After that, sequences of such patterns are used for interpretation, prediction, or controlled generation. In this sense, the issue is not merely a new tool, but a more general methodological move: complexity may be more productively described not by enumerating variants as such, but by identifying the abstract structures that organize those variants and make their further development predictable.

Any stronger generalization would still be premature, and for this reason the thesis does not formulate from this a universal law. Even so, the two projects together already support a more cautious but still substantial conclusion. Modern AI systems make it practically feasible to cross between two phases of working with a complex system: first extracting stable structure from observable variety, and then using that structure as the basis for interpreting new cases and producing new output. If this line is confirmed in other domains as well, then what is at stake may be not only a set of successful engineering

solutions, but the beginning of a new way of doing research on complex systems.

Chapter 6

System Architecture and Implementation

6.1 System Overview

The implemented ELA system spans backend processing, runtime orchestration, storage, media handling, and a learner-facing frontend. At a high level, the architecture is a pipeline from authentic input to explainable output.

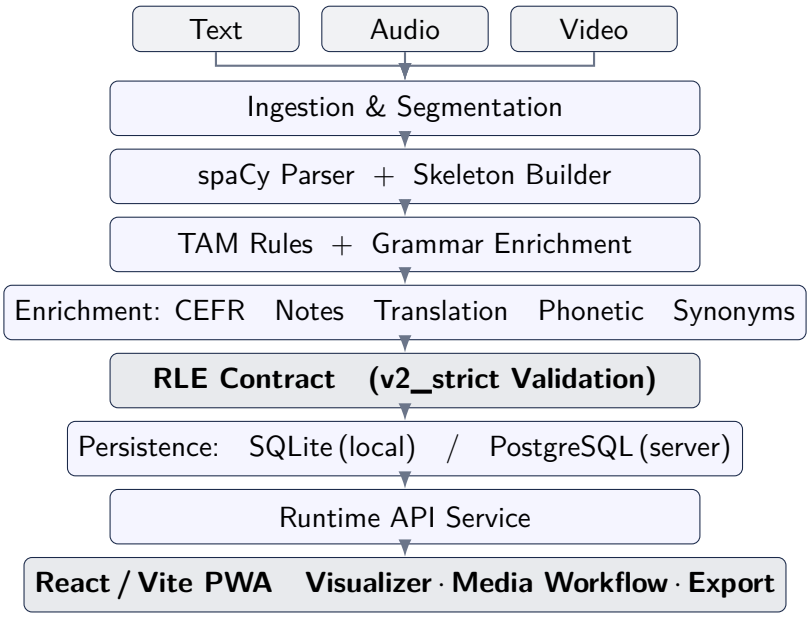

Figure 6.1: High-level architecture of the implemented ELA system

6.2 Technology Stack

Table 6.1: Representative technology stack of the current ELA implementation

Layer	Main technologies	Role in the system
Frontend	React, Vite, TypeScript, PWA packaging	learner-facing workflow, visualizer, file/project interaction
Desktop packaging path	Tauri	desktop-oriented delivery path alongside the web application
Runtime/API	Python runtime service, HTTP bridge	sentence-contract processing, media submission, job orchestration
Linguistic core	spaCy, deterministic skeleton builder, TAM rules	structural truth layer
Generated enrichment	controlled note-generation path, template transformation, optional T5-based rendering	user-facing pedagogical wording
Translation and speech	M2M100, Whisper, espeak-ng, ffmpeg-related media tools	translation, ASR, phonetics, subtitle/media processing
Storage	SQLite and PostgreSQL paths	local state, persisted analysis history, reproducibility
Deployment	Docker Compose, Nginx	service isolation and reproducible deployment boundaries

6.3 Backend Linguistic Pipeline

The core pipeline is implemented in Python under the `ela_pipeline` package. The project README and module structure indicate a rules-first design with the following major stages:

1. input ingestion and segmentation;
2. parser-backed skeleton construction;
3. deterministic TAM enrichment;
4. optional CEFR, note, translation, phonetic, and synonym enrichment;
5. contract validation;

6. persistence and export.

One of the most important architectural decisions is that structural truth is not delegated to a generative model. Parser output, rule logic, and validators form the trust boundary, while generated content is treated as controlled enrichment.

Table 6.2: Rules-first ownership in the implemented ELA pipeline

Layer	Main responsibility	Why this matters
spaCy + skeleton builder	structural graph, sentence/phrase/word hierarchy, POS, dependency-backed spans	keeps core representation deterministic and inspectable
TAM rules	tense, aspect, mood, voice, finiteness, TAM construction logic	prevents free-form generation from inventing grammar labels
Tabular or classifier path	CEFR-oriented predictive enrichment and blueprint support	separates classification from free-form wording
Controlled renderer / note path	lexical realization of learner-facing note text	constrains generation to wording rather than structure
Validator	contract safety, schema enforcement, field ownership boundaries	enforces architectural discipline across all later stages

6.4 Runtime Service and API Layer

Beyond offline inference scripts, the current system includes a dedicated runtime layer responsible for project management, client storage, media submission, background processing, UI state, and an HTTP bridge for the frontend. This is a significant step beyond the earlier thesis, which focused more on conceptual architecture than on operational service boundaries.

The runtime layer also reveals one of the current weaknesses of the project: the service has become strict enough to expose inconsistencies between expected frontend payloads and actual sentence-node shapes. This makes the runtime layer important not only as a technical mediator between components, but also as a practical test of the architecture itself. It is at the runtime level that one can see whether the contract, the pipelines, multimodal processing, visualization, and user actions are actually capable of functioning as parts of one system rather than as a collection of separate experiments.

In its actual structure, the runtime API should therefore be treated not as an internal helper layer, but as a real product boundary. The project documentation and the implementation define routes for project creation and selection, file registration, media submission for processing, tracking of background jobs, retrieval of data for the visualizer, and application of user edits. This matters for the present thesis because it shows a significant shift in the level of implementation: the work did not stop at producing structured JSON output, but progressed to a service layer through which a real end-to-end learner workflow can operate.

Table 6.3: Representative runtime/API responsibilities described in the current project documentation

Runtime concern	Role
Project and file state	manage learner projects, selected project state, and media registration
Sentence-contract path	expose text-to-contract processing through HTTP endpoints
Media submission	route uploads through local processing or policy-based rejection paths
Background job handling	support long-running tasks and status polling for media workflows
Visualizer payload delivery	provide contract trees back to the frontend in UI-facing form
Edit and synchronization layer	preserve local edits, retry paths, and resume-oriented workflow state

6.5 Frontend

The frontend is implemented as a React/Vite progressive web application. It organizes the user experience around projects, files, analysis, vocabulary, configuration, and a linguistic visualizer. Compared with the earlier Kivy-centred prototype framing, this frontend reflects a move toward a more modern web-oriented delivery model and a clearer separation between presentation and processing.

The frontend development note is particularly useful because it records the practical scope of what was actually implemented. It confirms a project-first flow of 'Projects -> Files -> Analyze', project-scoped file registration, stage progress reporting, artifact download actions, sentence-by-sentence visualizer navigation, translation-provider handling, quick node edit controls, and vocabulary export. It also records an important architectural subtraction: backend media-job UI controls were removed after the client-

first runtime decision. That kind of removal is as important as any added feature because it shows that the final architecture was shaped by narrowing responsibilities, not only by accumulating them.

6.6 PWA and End-to-End Workflow Evidence

In the present work, the PWA (Progressive Web Application) is a web application that the user opens in a browser but can also install on their device in much the same way as an ordinary application. For this reason, it combines the properties of a web interface with those of a more persistent application environment: it preserves local state, uses browser background mechanisms for more stable operation, and supports a more coherent mode of interaction with the system. In the context of ELA, this PWA should therefore be seen not simply as an interface placed on top of separate backend functions, but as the real user environment in which the main capabilities of the system are expected to work together.

It is through the PWA that the user creates a project, uploads files, launches analysis, receives processing results, obtains learning artefacts such as transcripts, translations, subtitles, and other derived materials, and interacts with the visualizer. For this reason, verification of this part of the system is especially important for the thesis: it shows whether linguistic analysis, multimodal processing, runtime logic, and the user workflow were actually brought together into one working system.

To verify the PWA, a dedicated integration scenario was used during the work, covering the full user path: opening the application, checking the readiness of the built-in browser mechanisms, creating a project, uploading a real media file, opening the analysis screen, running the pipeline from transcription through rendering, and obtaining the final artefacts. This is important because such testing moves evaluation beyond isolated functions and shows how the system behaves under real browser-based end-to-end conditions.

It is equally important that work on the PWA exposed a number of practical engineering problems that could not have been seen at the level of isolated modules: the readiness of locally used models, the correct connection between the user interface and the backend service, problems caused by outdated cached versions of the application in the browser, and the need to replace one method of drawing text onto video with another that proved more stable for the real user scenario. In this sense, the PWA turned out to be not only the user-facing shell of the project, but also the environment in which the real integration maturity of the whole system was tested.

6.7 Media Processing

Media processing in ELA did not develop along a straight line, but through a revision of the project's initial architectural expectations. At one stage, it was assumed that a substantial part of interactive processing could be performed directly on the user's device inside the PWA. This line also received practical implementation: local frontend workers were introduced for speech recognition and translation, designed to operate with locally available models in the browser environment. At that point, the project was genuinely moving toward stronger client-side processing.

As the system developed further, however, it became clear that such a scheme could not serve as the sole basis for the entire multimodal layer. There were several reasons for this. First, local processing depended too heavily on whether the necessary models were available on the user's device and whether that device had sufficient computational resources for stable work with real audio and video files. Second, the full media workflow already included not only recognition and translation, but also the construction of contract-native sentence nodes, the persistence of document artifacts, the generation of several subtitle variants, TTS, and the export of final files, all of which required a more centralized and reproducible orchestration layer. Third, as the pipeline became more complex, it became important not merely to obtain an intermediate result in the browser, but to support reuse of already completed stages, stable linking between sentences and contract nodes, and reliable recovery after partial failures.

For this reason, the project gradually shifted toward a different organization of processing. Media submission remained part of the user-facing PWA workflow, but the main stable pipeline was moved into the runtime/service layer. It is there that source-type detection, text or transcript extraction, sentence-stream construction, conversion into contract-native sentence nodes, document-artifact persistence, subtitle generation, TTS, and export of final media files are carried out. In other words, local processing did not disappear without trace, but it ceased to be regarded as a sufficient foundation for the complete working pipeline.

This architectural turn is itself important for the thesis. It shows that multimodality in ELA was not simply a matter of "adding audio and video," but became a point at which the boundaries between the browser, the user's device, and a separate processing layer had to be redefined. The practical result is that the media side of the system took shape as a hybrid architecture, but with a clear shift toward runtime/service orchestration rather than toward fully local execution of the entire pipeline on the user's device.

6.8 Persistence and Export

In the present work, data persistence was organized according to a client-first principle. This means that the main user workspace is located on the client side, that is, on the user's own device, rather than being fully moved into an external backend. This decision was important not only technically, but also conceptually: ELA was designed as a system in which the user should retain control over projects, files, analysis results, and derived learning materials without being forced to depend on permanent remote storage.

For this reason, the PWA preserves the main entities of the user's work: projects, the currently selected project, uploaded files, analysis history, processing settings, and final results. This also includes the derived learning artefacts that appear after analysis, such as text representations, contract structures, translations, subtitles, and other materials that the user can later view, reopen, and use beyond the moment of immediate processing. In practical terms, this makes the system not merely a tool for one-time analysis, but a working environment in which results retain value after the completion of a particular pipeline run.

Against this background, the backend takes on a different role. It is needed not as the main storage layer for user data, but as the processing layer in which the more computationally heavy and model-intensive stages of the pipeline are carried out. It is here that the ASR, translation, CEFR, note-generation, and related LLM-based or ML-based components used in the project perform their work: text extraction and transcription, construction of contract structures, grammatical and classificatory processing, translation, generation of pedagogically meaningful artefacts, subtitles, speech synthesis, and final media outputs. However, once these stages are completed, the results do not remain primarily in the backend layer, but are returned to the client environment and preserved there as part of the user's workspace. In this way, the backend concentrates computation and model-based processing, but does not replace the place where the user's own data actually live.

This division of roles became an important architectural outcome of the project. It made it possible at the same time to preserve the local ownership of user data, support reopening and continuation of work, and provide export of results in explicit external formats. Because of this, ELA supports not only the internal execution of analysis, but also reproducibility, resumability, and the use of results outside the current application session. In this sense, persistence and export in the present work are not a secondary technical detail, but part of the overall logic of the system: the user's workspace remains with the user, while model-based processing and the formation of complex artefacts are organized as a separate service layer.

6.9 Contract Safety as an Architectural Principle

One of the most important architectural decisions in the project was the requirement of contract safety. Its meaning is that the ‘RLE contract‘ should not be treated as free-form textual output from a generative model. On the contrary, the ‘RLE contract‘ serves as the structural foundation of the system, that is, as an already formed description of the sentence onto which later stages of interpretation and linguistic realization may be applied. In this logic, the generative component does not create the contract anew, but operates on top of an already constructed ‘RLE contract‘.

From this there follows directly a principle of field ownership. A generative model may participate in the formulation of user-facing explanations, notes, and other textual outputs, but it must not overwrite those fields that express the structural and grammatical foundation of the system inside the ‘RLE contract‘. These include node identifiers, links between nodes, source spans, syntactic dependency fields, ‘grammar_classes‘, CEFR-related predictions, and other parts of the contract that must remain under the control of parser-based, rules-based, or classifier-based layers.

For this reason, contract safety matters in the present work not only as a technical restriction, but also as a way of making the system genuinely explainable and verifiable. Explainability here is ensured not only by how the result appears in the interface or how convincing its wording may be, but above all by the fact that different parts of the ‘RLE contract‘ belong to different sources of formation and are not allowed to collapse into one another. User-facing text may be generated, but the structural basis of the sentence, its grammatical relations, and its classificatory features must not be altered arbitrarily at the generation stage. It is in this sense that contract-safe architecture became one of the strongest engineering ideas in the project: it makes explainability a property not only of the presentation layer, but of the internal organization of the system itself.

6.10 Deployment and Service Boundaries

Code development in this project followed a TDD-oriented workflow, whereas deployment was treated as a separate operational procedure for bringing ELA into a stable production state. In this work, the production instance of the system was deployed on the Hetzner-hosted server `el-a.uk`, where a server-side copy of the repository was maintained in `/opt/ela` on branch `master`. The deployment procedure was explicitly defined: changes were first merged locally from `dev` into `master` and pushed to the remote repository, after which the server executed `git pull` followed by a Docker Compose restart. As a result, the production system was updated not through a separately published build artifact, but through rebuilding the server instance directly from the current repository state.

This procedure makes it clear that by the final stage of the project ELA existed not

merely as a collection of research scripts, but as a system with explicit service boundaries, technical constraints, and a formalised update procedure.

Infrastructure roles:

- the frontend is responsible for the user interface;
- the application layer performs the core system logic;
- PostgreSQL provides persistent data storage.

Constraints:

- the production environment is built in a dedicated container configuration rather than matching the full development environment;
- not all components are permitted to be downloaded at runtime;
- part of the system behaviour is predefined through environment variables and deployment mode.

Update procedure:

- changes are first merged into `master`;
- the server then pulls the current repository state;
- the containers are rebuilt and restarted afterwards;
- the frontend is updated with a forced rebuild without cache.

Chapter 7

Dataset and Corpus Engineering

7.1 Why Dataset Engineering Matters

In this work, dataset engineering was necessary because for ELA it was not enough simply to collect a large corpus of English texts. The main task was to convert heterogeneous examples into controllable training material suitable for specific system components. In practice, this required `deduplication`, the introduction of `target caps`, template-based normalisation, the extraction of `template slots`, the construction of `placeholder pairs`, and explicit coverage control across grammatical and pedagogical categories.

One of the central tasks was balancing. The raw data contained a large number of repeated or near-repeated target formulations, which meant that a model could easily overfit to the most frequent patterns. For this reason, the work introduced limits on identical or almost identical targets and reduced redundant series of examples that did not contribute new training value. This made it possible to keep the dataset in a more balanced state and to prevent a small number of high-frequency formulations from dominating the rest of the material.

Template-based normalisation was equally important. For note generation, the goal was for the model to learn stable pedagogical constructions rather than arbitrary surface-level explanation texts. Accordingly, similar explanatory outputs were mapped into more stable template forms. Instead of treating every phrase as a unique target, the data were progressively reduced into a more compact space of reusable explanatory patterns. This reduced noise, improved quality control, and made note generation more predictable.

The next step was parameterisation. A template alone is not sufficient if it remains rigidly tied to one specific sentence. Therefore, the work used `template slots` and `placeholder pairs` to separate stable explanatory structure from the variable elements of a sentence. This allowed the same note pattern to be associated with different lexical fillings and grammatical realisations. The approach was especially important for the contract-oriented logic of the project, where what mattered was not only the surface text

of an example, but also the structural relations and grammatical features underlying it.

Finally, dataset engineering in the project involved continuous coverage control. It was not enough merely to clean the data and reduce repetition; it was also necessary to verify which grammar classes, CEFR levels, and note target types were actually represented in the dataset. Otherwise, even after cleaning and deduplication, the result could still be a corpus that covered only the most frequent cases while remaining weak on rarer but substantively important constructions. For this reason, the datasets in this work were built not as passive stores of examples, but as deliberately regulated material in which balancing, template-based normalisation, parameterisation, and coverage control were necessary conditions for training, interpretability, and further system expansion.

7.2 Source Families

During the work, dataset construction was based on several different source families rather than on a single corpus or a single textbook. In the last stage of the project, the source layer was expanded through online resources suitable for further structural filtering and projection. The working set was extended with Tatoeba, Wikinews, Simple Wiktionary, and raw sentence material from UD_English-EWT and the merged UD_English-EWT + UD_English-GUM layer. In addition, projection_corpus_external_v1 was assembled not as an arbitrary external text archive, but as a prepared sentence set intended for further normalisation, deduplication, and grammar-oriented projection. As a result, the source layer of the project came to include not only curated corpora and treebanks, but also online textual resources used as providers of raw sentence material.

This expansion is reflected in the current project artefacts. The file ud_merged_treebank_sentences.report.json records 26,416 sentences after merging UD_English-EWT and UD_English-GUM; of these, 14,586 rows came from EWT and 11,830 from GUM. The GUM layer contributed not only additional sentences, but also genre variation, including academic, court, essay, news, speech, and textbook. The file projection_corpus_external_v1.report.json records 28,195 final rows assembled from ingested_sentences.jsonl and the merged UD layer. The distribution by source in this set is as follows: 13,395 rows from ud_ewt_treebank, 11,827 from ud_gum_treebank, 1,190 from tatoeba, 1,190 from ud_ewt, and 593 from wikinews. In addition, the repository now contains simplewiktionary-latest.xml.bz2, meaning that the Wiktionary layer also entered the project as an actual source asset rather than as a merely planned resource.

At the same time, the book-derived branch of the work was also expanded. In addition to the dictionary and instructional sources already used earlier, the note-oriented pipeline was extended through newly processed books and manuals. This set included *Basic English Grammar* by Betty Azar, *Introducing English Grammar* by Kersti Börjars and Kate Burridge, *English Grammar Drills* by Mark Lester, *Mysteries of English Grammar* by An-

dreea S. Calude and Laurie Bauer, *Explaining English Grammar* by George Yule, as well as a selected-book subset comprising *The Grammaring Guide to English Grammar* by Peter Simon, *English Grammar For Dummies* by Geraldine Woods, and *English Grammar Demystified* by Phyllis Dutwin. A separate open-access branch was also formed, containing Steven Brehe's *Brehe's Grammar Anatomy* and Randal Thiessen's *English Grammar for Academic Purposes*. Thus, the work relied not only on corpora and treebanks, but also on several independent book-derived pipelines aimed at extracting explanatory note pairs, phrase-level examples, and template-friendly pedagogical material.

The scale of this work is visible in the cumulative results of book-source processing. The file `processed_books_cumulative_summary_source_first_v6.json` records `11` book families, `15,791` raw pairs, `13,996` clean pairs, and `8,412` final rows after later row selection. The largest contributions came from `Oxford Dictionary` (`2,746` rows), `Cambridge Dictionary` (`2,222`), and `English Grammar Today` (`1,835`), but the set was not limited to them: it also included `Leech Glossary` (`496`), `GSWE` (`466`), `COBUILD 2017` (`186`), `English Grammar in Use` (`117`), `English for Everyone Practice` (`120`), `Collins COBUILD 2011` (`130`), `Oxford Handbook` (`85`), and `Azar Basic Grammar`. In other words, a large quantity of heterogeneous source material was genuinely processed during the work, and the bottleneck did not lie in the absence of input material.

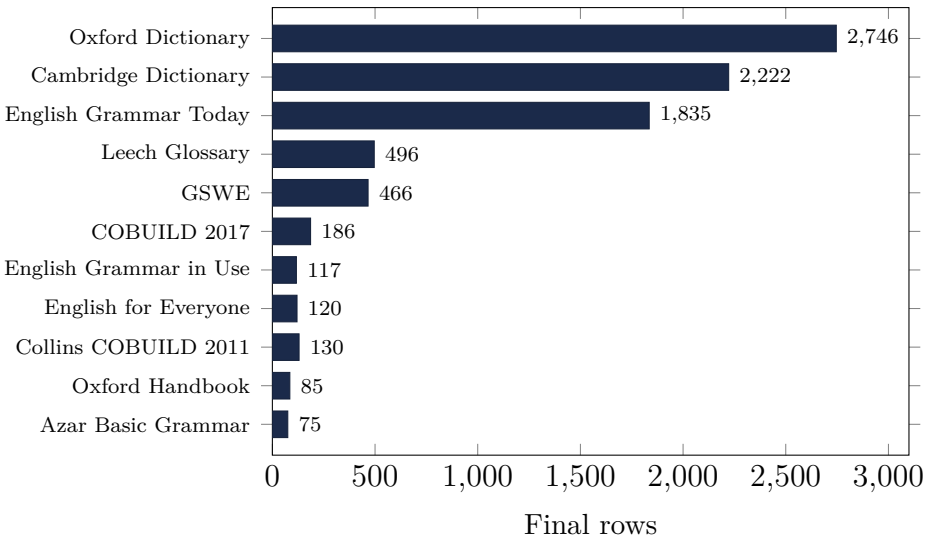

Figure 7.1: Final row counts per book source in the cumulative training corpus (v6 summary). Oxford Dictionary, Cambridge Dictionary, and English Grammar Today together account for over 80% of the dataset.

However, this stage proved to be both the most labour-intensive and the least efficient part of the entire project. Despite the variety of corpus-based, dictionary-based, and book-derived sources, the final outcome was only about `160+` unique `notation -> sentence` pairs suitable for further controlled use. This does not mean that the listed sources lacked sufficient material for the project. On the contrary, the work showed that

transforming rich instructional and reference material into a compact set of genuinely usable notation-sentence pairs required disproportionately large amounts of time and effort. At each stage, the process involved extraction, cleaning, filtering, alignment, templating, deduplication, and repeated row selection, while a substantial part of the source material had to be discarded as noisy, repetitive, overly lexicalised, or poorly transferable into a template-oriented format.

For this reason, the present stage does not allow the thesis to demonstrate broad and convincing coverage of the note-generation space at the level that would be needed for strong general claims about the completeness of this subsystem. The diversity of sources should therefore be interpreted not as evidence of already achieved coverage, but as evidence of how expensive and slow the extraction of high-quality pedagogical signal from real linguistic and instructional material turned out to be.

7.3 Curation and Stabilization

During the work, curation and stabilisation were tied above all to the attempt to bring the `RLE contract`, the note pipeline, and the training pairs into a state in which the system could operate predictably and verifiably. At this stage, however, it became clear that the main limitation was not only code quality or schema design, but also the structure of the available training material itself. Despite the large number of processed sources, the final number of genuinely usable unique `notation -> sentence` pairs remained small, while the construction of reliable `notation -> phrase` and `notation -> word` pairs proved to be even more difficult and costly.

For this reason, a compromise decision was made at this stage to restrict note-oriented training and stabilisation primarily to `notation -> sentence` pairs. This did not mean that phrase-level and word-level note logic were considered unnecessary. On the contrary, the attempt to build such pairs revealed how difficult it was to obtain sufficiently clean, non-redundant, and structurally compatible material for them. At the phrase and word levels, the problems of noise, poor transferability of note text, excessive lexicalisation, boundary ambiguity, and weak repeatability of explanatory patterns increased sharply. As a result, these pairs required disproportionately large amounts of effort in extraction, alignment, filtering, and templating, while failing to provide a comparable gain in stable training signal.

It was in this context that the formalisation of a `v2`-compatible `RLE contract` became especially important. A stricter `RLE contract` was needed not simply as an exercise in architectural neatness, but as a way to keep the sentence-level pipeline controllable after stepping back from broader note targets. If the system at this stage was to rely mainly on `notation -> sentence` pairs, then the sentence structure, node inventory, node relations, and admissible fields had to be defined with greater precision, because

the sentence layer had become the main carrier of note-generation signal.

The strengthening of fallback behaviour for notes followed from the same constraint. When the number of genuinely usable unique sentence pairs is small, dependence on unstable generative output becomes especially risky. Stabilisation therefore required not only improving model outputs, but also making the note layer safer: if a generated note turned out to be weak, noisy, or poorly aligned with the `RLE contract`, the system had to provide a more reliable fallback path. Otherwise, even a modest shortage of high-quality training pairs would quickly propagate into instability across the entire note-oriented pipeline.

The tightening of validation modes and schema enforcement was also a direct consequence of this compromise narrowing of scope. Once the training and project focus shifts toward sentence-level pairs, it becomes especially important not to let partially plausible but actually incompatible outputs continue through the pipeline. In this work, that meant stricter checks to ensure that the `RLE contract` remained structurally coherent, that note-related fields did not drift away from the actual form of the sentence, and that downstream stages did not continue operating on data that had already fallen outside the acceptable boundary.

The expansion of test coverage and the synchronisation of documentation with implemented behaviour were part of the same stabilisation step. After the compromise decision had been made, it became necessary to state explicitly what was in fact supported as a note target at this stage, which limitations had to be treated as real, and which directions remained unfinished. Without this, the project would continue to look as if sentence-, phrase-, and word-level note pairs were progressing equally, whereas the actual work showed the opposite.

Stabilisation at this stage therefore did not mean declaring the system broadly "ready." It meant narrowing the working scope more strictly to what had actually been made manageable. Because of the small number of unique `notation -> sentence` pairs and the even greater difficulty of constructing high-quality `notation -> phrase` and `notation -> word` pairs, the project had to rely on sentence-level note training as a compromise baseline that was limited but practically achievable. This decision reduced the breadth of the current note-generation layer, but at the same time made it more honest, more testable, and more suitable for further development.

7.4 Deduplication and Quality Control

During the work, it became clear that for ELA an increase in row count by itself did not guarantee high-quality training material. The main problem was that larger datasets quickly became saturated with repeated note targets, weak explanatory formulations, and structurally low-value variants that increased data volume without providing a compa-

rable gain in training signal. For this reason, deduplication and quality control became not a secondary cleanup step after data collection, but one of the central mechanisms of the entire dataset-engineering process.

In practice, this required several successive constraints. Repeated targets had to be filtered and capped so that the model would not overfit to a small number of high-frequency note formulations. At the same time, low-quality, noisy, or telemetry-like notes were excluded because they did not provide a full pedagogical explanation. After that, it was necessary to ensure that cleaning did not destroy CEFR balance or grammar-class coverage. In addition, note suitability and trace fields were checked, meaning that the dataset preserved not just any rows, but only those that could be reliably linked to their source, their contract structure, and their later template-oriented processing.

The result of this policy was not a minimal dataset, but a stabilised working dataset layer in its latest form. For Experiment A, which tested whether a richer RLE representation improves note selection, the project used a contract-input raw-note dataset of `3875` rows after balancing, with `58` unique raw note targets. For Experiment B, which compared the dynamics of raw and templated notes, the project used a large mixed paired-template dataset of `5016` rows, containing `3545` templated targets and `1471` raw targets. These dataset sizes are documented in the repository report and dataset artifacts listed in the bibliography. They better represent the state to which the dataset layer had actually been brought after the major stages of cleaning, balancing, and rebuilding, rather than earlier noisier or more aggressively compressed intermediate variants.

This matters because these figures show the real outcome of the compromise between scale and controllability. On the one hand, the project did not stop at very narrow experimental samples of only a few hundred rows. On the other hand, it did not try to preserve maximum size at any cost. Instead, the dataset layer was brought to a state in which several thousand rows could already function as a more disciplined and structurally controlled body of material: with repeated targets constrained, with clearer traceability, with raw and templated targets separated, and with a more interpretable distribution across the note space.

For this reason, deduplication and quality control in the project should be understood not as cosmetic post-processing, but as a mechanism of structural discipline over the datasets. The goal was not simply to obtain as many rows as possible, but to obtain datasets in which scale no longer destroyed signal quality. The final working datasets of `3875` and `5016` rows should therefore be treated as the outcome of deliberate engineering selection rather than as a passive by-product of data accumulation.

7.5 Placeholder Construction

During the work, it became clear that the note-generation path in ELA could not be built as direct training on ready-made final explanatory texts. A key step was the conversion of enriched linguistic structures into more abstract template forms with placeholders. It was precisely at this point that dataset engineering became directly connected with the central research logic of the project: the pedagogical signal had to be derived not from the accidental wording of note text, but from a repeatable structural pattern that could later be re-filled from the `RLE contract`.

This decision was made explicit in the canonical process document dated `2026-03-21`, where the whole pipeline was described as a sequence of linked stages. First, `notation + content` pairs were extracted from book-derived sources. Then a contract-tree representation was built from the content. After that, the notation was analysed and converted into placeholder-based template text. The placeholders were then constrained to attributes actually available in the corresponding contract. Training was performed not on fully rendered final notes, but on templated notation. At inference time, a new contract was rebuilt for new content, the model was expected to output a template with only allowed placeholders, those placeholders were then filled from the contract, and only after that was the final rendered note written back into the tree. In other words, placeholders were not used as a local text-formatting trick, but as the central mechanism linking dataset construction, model training, and runtime rendering.

This is visible in the actual data regimes used in the project. One regime used a templated input with an ordinary, non-templated output. For example, an input could take the form `{{SUBJECT}} {{MODAL}} {{BASE_VERB}} {{NOUN_PHRASE}}`, while the target remained an ordinary pedagogical note such as "May can present a future event as possible rather than certain." In this mode, the model learned to map an abstract sentence pattern to a finished explanation without placeholders in the output itself.

A stricter regime used placeholders on both sides. For example, an input could take the form `{{SUBJECT}} {{AUXILIARY}} {{BASE_VERB}} {{IF_CLAUSE}} {{SUBJECT}} {{BASE_VERB}} {{OBJECT}} {{ADVERB}}`, while the output was itself templated: "In the first conditional, the `{{IF_CLAUSE}}` uses the present simple while the main clause presents a future result." In this case, abstraction was performed on both sides of the pair: both the sentence and the explanation were converted into slot-based form for later controlled rendering.

There was also a third regime in which the input was not a sentence template but an already expanded contract-oriented payload, while the output remained an ordinary note text. This was the regime used in the contract-input experiments. For example, the input began as `task: rewrite_linguistic_note_template_from_contract_template payload: {...}` and contained information about node type, grammatical role, chil-

dren summary, sentence context, and the selected template, while the target remained an ordinary explanation such as "A conditional clause presents a possible or typical situation together with its consequence." In this case, placeholders could be absent from the output itself, but the input representation had already been restructured around structural abstraction rather than surface sentence form.

Finally, there was a transitional regime in which contract input was combined with template-based selection logic, while the final rendered explanation was still stored as ordinary note text. That is, the system already operated through template selection and allowed slots, but the final target in the training example could remain verbal and non-templated. This matters because it shows that placeholder construction in the project was not a single uniform scheme. It existed as several connected regimes of abstraction, ranging from template input with raw output, to template input with templated output, to contract-driven input with rendered explanatory output.

The later slot-normalised projection artefacts show that this strategy was brought to the level of a measurable transformation stage rather than remaining a conceptual idea. In the `v17` slot-normalised projection report, the repository records `3,505` projected rows. Within that volume, it separately records `1,984` sentence-level slot-templated rows and `6,584` phrase-level slot-templated rows, while also explicitly tracking cases of `lexicalized-note repair`. This is important because it shows that placeholder construction was carried out as a distinct procedure over a large projected corpus, with its own metrics, its own repair cases, and its own quality control. Placeholder construction in this work was therefore not a secondary text-improvement technique, but one of the central operations through which pedagogical note signal became trainable and reusable.

7.6 Template Opportunity Analysis

The history of the pipeline reports in the project shows that template expansion developed unevenly. At an earlier stage, after slot-normalised projection had already made a fairly large grammar space observable, the template inventory still covered that space weakly. This is especially clear when comparing an earlier template-opportunity snapshot with a later one. In both cases, the projection layer recorded the same `513` exact sentence families, but the number of sentence families for which book slot templates already existed increased from `6` to `202`. At the same time, the number of families that already had book notes but still lacked slot templates fell from `199` to `3`. These are the closed opportunities: the project succeeded in converting a substantial part of the sentence-level pedagogical material from the state "a note exists" into the state "a slot template exists."

This transition shows that template expansion in the project was not a decorative refinement, but a distinct technical stage. The system initially already contained notes for many sentence families, but these notes had not yet been turned into reusable slot

forms. The later reports show that a major part of the work consisted precisely in closing that gap: not merely extracting a note, but transforming it into a template that could be reused over the projection layer and then passed further into the controlled-rendering pipeline.

At the same time, the same report history shows the open opportunities. At phrase level, progress remained much weaker. Out of `96` exact phrase families, the number of families with book notes rose only to `10`, while the number with fully formed slot templates rose only to `3`. Consequently, even after template-layer expansion, `7` phrase families still had notes but no slot-template coverage. This means that by this stage the main unresolved area of the project had shifted clearly toward phrase-level templating.

Substantively, these open opportunities did not cluster at random. The reports repeatedly brought to the top phrase families connected with relative-clause-heavy material: restrictive and non-restrictive relative patterns, phrases with embedded relative structure, and related constructions where note transfer remained unstable. It was precisely here that the template inventory lagged most clearly behind the observed projection space. In other words, if on the sentence level the project had largely learned how to close the gap between extracted notes and reusable slot templates, then on the phrase level that same gap remained largely open.

An additional layer of this history is that not all sources progressed toward slot templating equally. Some book-derived reports continued to record mainly `passthrough` outputs, that is, notes that were usable as notes but had not yet been transformed into parameterised slot form. This is also important for interpreting the dynamics: even when note extraction had already been completed, a template opportunity could remain open if the material never passed through the next stage of formalisation.

The history of the pipeline reports therefore reveals two distinct lines. The closed opportunities were primarily sentence-level families, for which the project managed to convert book notes into slot templates at scale and to reduce the number of uncovered note-to-template gaps sharply. The open opportunities were mainly phrase-level families, especially those involving heavy relative-clause patterns, where notes were either absent or did not transition into stable slot-based form. For this reason, template opportunity analysis in the work functioned not as summary statistics, but as a historical diagnostic instrument: it showed which parts of the template space had already been disciplined and which still remained the principal source of further work.

7.7 Measured Template Space

During the work, the template inventory ceased to be treated as an informal collection of useful note patterns and was instead turned into a measurable object. This meant that the project evaluated not only the existence of individual templates, but also their

quantity, distribution, and typological composition across already constructed template-oriented and note-classification datasets. Without this step, any discussion of template diversity would have remained merely descriptive: one could claim that templates were "many" or "few," but not show what template space had actually been assembled and used by this stage of the project.

One of the clearest views of this space came from a sentence-multilabel template dataset built over the covered projection layer. After filtering, it contained 1774 rows, 48 total template identifiers, 42 sentence template types, and 6 phrase template types. These figures matter because they describe not simply the size of a dataset, but the structure of the template inventory itself. By this stage, the template layer could already be described not only through isolated successful examples, but through template types, their counts, and their distribution across sentence-level and phrase-level branches.

More importantly, however, this measurable template space also appeared in the final working datasets. In the contract-input raw-note dataset used in one of the final experiments, 3875 rows remained after balancing, with 58 unique raw note targets. This dataset showed that even when richer RLE input was used, the project was already operating over a sufficiently large and structurally organised note space in which note-selection behaviour could be measured against a stable set of target classes. The note-target space here was no longer arbitrary; it existed as a bounded and observable inventory suitable for classifier-style evaluation.

The picture became even clearer in the mixed paired-template dataset. This final working dataset contained 5016 rows, of which 3545 belonged to templated targets and 1471 to raw targets. It was here that the measurable template space appeared in its most direct form: the project could compare two target regimes inside one dataset layer, one built around raw note targets and the other around templated targets. In this way, the template space in the project was no longer treated as a by-product of note writing, but as an independent axis of dataset design along which target volume, target types, and model behaviour could be studied separately.

Taken together, these three views produce an important picture. The dataset of 1774 rows showed the internal form of the template inventory as a set of template identifiers. The dataset of 3875 rows showed how a structured note space functioned in contract-input note classification. The dataset of 5016 rows showed how raw and templated target regimes related to one another inside one final working layer. In other words, by this stage the project was no longer merely using templates, but measuring template space repeatedly in different projections: as inventory types, as note-class targets, and as competing raw-versus-templated regimes.

At the same time, these same figures also show the limitation of the current stage. Template space had become measurable, but not balanced. The dataset containing 42 sentence template types and only 6 phrase template types already makes clear that the

sentence-level branch was substantially more developed than the phrase-level branch. Likewise, the split inside the 5016-row final dataset into 3545 templated and 1471 raw targets shows that the templated regime had become the dominant branch, but had not displaced the raw-note layer completely. For this reason, measured template space in this work should be understood not as a completed map of templates, but as the first sufficiently strict quantitative form of a template space that the project had begun to treat as a separate object of analysis.

7.8 From Free-Form T5 to Controlled Rendering

During the work, I gradually moved away from the early scheme in which T5 was expected to generate a complete explanation directly from the input representation of a sentence. In the later architecture, the process was reorganised. First, an RLE contract was built and validated, that is, a structural description of the sentence with its nodes, relations, and grammatical features. Then the system extracted from that representation the features needed to determine the construction type, the difficulty level, and the appropriate class of explanation. In one of the later branches, this was done not by a generative model, but by XGBoost classification over explicitly defined features: syntactic signatures, grammar classes, tense-aspect-mood information, node type, node role in the RLE tree, the selected template, and other contract fields. At that stage, the model no longer wrote the explanation itself. Instead, it selected what kind of explanation the sentence required. Only after that did the system move to the template and to the verbal formulation of the final note.

This was a principled change in the work. In the earlier scheme, the model tried to write the final explanatory text immediately, so too much depended on free generation. Even when the output sounded plausible, it could remain only weakly tied to the actual grammatical structure of the sentence. In the later scheme, I first fixed the structure of the sentence, then determined its grammatical characteristics, and only after that allowed the system to move to textual formulation. In other words, generation stopped being the place where the main decision about the meaning of the explanation was made. That decision was made earlier, at the level of structure, features, and explanation-class selection.

This was the main architectural shift. I did not remove models from the system, and I did not replace everything with rules. What I removed was free text generation from those stages where it introduced too many errors and distortions. As a result, the model no longer solved the task "invent an explanation from scratch," but a narrower task: to phrase an already selected explanation in words without breaking its link to the grammatical structure of the sentence.

7.9 How the Dataset Strategy Evolved: From Raw Corpora to Grammar Books

The dataset construction strategy reached its current form through several distinct phases, each motivated by a concrete limitation discovered in the previous one. Understanding this evolution is important because the final "closed" architecture is not an initial design choice but the result of iterative problem-solving.

Phase 1: Raw Corpora and Free-Form Generation

The earliest dataset version ingested open-licence corpora—UD English Web Treebank, OANC, Tatoeba, and Wikinews—producing around 3,000 sentences and their parsed representations. T5 was trained to output a free-form pedagogical note given a spaCy contract describing the sentence. Notes were unconstrained: the model could write anything. The variability of output appeared high, but inspection revealed that the model quickly converged on a small set of generic phrasings that were technically grammatical but offered little pedagogical value.

Phase 2: Grammar Blueprints

To stabilize output quality, the project introduced a library of over 150 *grammar blueprints*—hand-written, rule-based note templates keyed to specific grammatical classes such as `present_simple_affirmative`, `relative_clause`, or `passive_voice`. Each blueprint provided a fixed note at three CEFR tiers (elementary, intermediate, advanced). Blueprints served as a deterministic fallback when model output failed quality gates. The approach improved consistency, but the inverse problem became clear: the blueprint library itself was too coarse. The same note appeared thousands of times in the projected corpus, and the model had no incentive to learn anything except which blueprint to select.

Phase 3: Slot-Based Templates

The next step was to parametrize blueprints with named placeholders. Instead of a fixed string, a note template could contain `{{HEAD_NOUN}}`, `{{PREPOSITION}}`, `{{OBJECT_NP}}`, or `{{RELATIVE_MARKER}}`—fields resolved at projection time from the linguistic contract of the matched instance. This made each template potentially produce a distinct note for every sentence it matched. The technical infrastructure for this—slot validation, contract-to-slot resolution, template registry—was introduced in March 2026 and is described in more detail in the Placeholder Construction section above. The placeholder system worked, but exposed a new bottleneck: the repository contained only a few dozen

manually authored template variants. Corpus projection amplified those templates across many sentences, but the underlying pattern space remained narrow.

Phase 4: Grammar Books as the Source of Patterns

The diagnostic finding, documented explicitly in the template expansion strategy report for v16, was that *the main bottleneck is insufficient template granularity.* The solution was to stop authoring templates manually and instead extract them from existing pedagogical sources. Specialized grammar handbooks—Collins COBUILD English Grammar, George Yule's *Explaining English Grammar*, Betty Azar's *Basic English Grammar*, Mark Lester's *English Grammar Drills*, Farlex *Complete English Grammar Rules*, and others—contain thousands of author-written explanations, each paired with a curated example sentence. These are exactly the slot templates the project needed: validated by domain experts, covering a wide range of constructions, and already matched to example sentences that could drive the contract extraction step. A book extraction engine was built to parse EPUB, PDF, and DJVU formats and output the same notation–content pair format used throughout the pipeline.

Canonical Training and Inference Flow

The canonical process for book-to-dataset-to-inference, documented explicitly on 2026-03-21, fixes the full flow in ten steps. The first six steps produce the training dataset; the last four describe what happens at inference time for a new sentence.

Training path (steps 1–6). A grammar book provides pairs of *notation* (the author's pedagogical explanation) and *content* (the example sentence). The content is parsed by spaCy into a linguistic contract tree. The notation is analysed and converted into a *template*: the same explanation text, but with content-specific terms replaced by named placeholders such as `{{HEAD_NOUN}}` or `{{PREPOSITION}}`. Placeholders may reference only attributes that exist in the contract built from the paired content—the contract is the sole source of allowed slot values. Each book pair then becomes one T5 training row: the input is the contract context together with the template structure; the *target is the template text with placeholders intact*, not the rendered final note.

Inference path (steps 7–10). When a new sentence arrives from a user, it passes through the same spaCy parser, producing a contract with the same schema as the training contracts. The T5 model, trained on templated targets, outputs a notation template with placeholders. Those placeholders are then filled from the very same contract that was the model's input—the contract serves double duty as both the generation context and the slot-value source. The filled template is the rendered pedagogical note, saved back into the contract tree under `linguistic_notes`.

This symmetry—the contract is the context at training time, and the same contract schema provides the slot values at inference time—is the technical foundation of controlled note generation. The model never sees raw unbounded text as a target and never invents structure; it only realises wording for a template derived from and constrained by the contract.

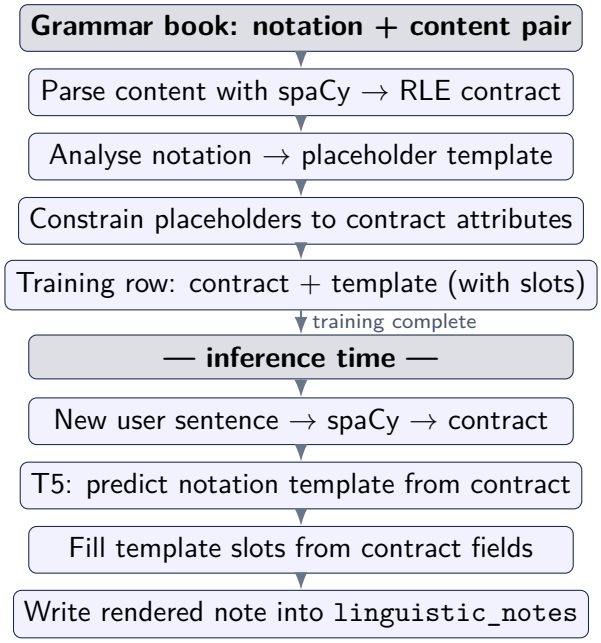

Figure 7.2: Closed pipeline from grammar-book source to inference-time note rendering. Steps 1–5 produce training data; steps 6–9 describe inference for a new sentence.

Field Ownership

The architecture formalises ownership of every contract field across three layers, as fixed in the field ownership document dated 2026-03-04:

- **spaCy and deterministic rules** own all structural fields: the tree shape, node types, part of speech, dependency labels, morphology, TAM fields (tense, aspect, mood, voice, finiteness), grammar classes, and note blueprints. These fields are set once and cannot be modified by any downstream model.
- **Tabular ML (XGBoost)** owns `cefr_level`. This is the only field produced by a trained classifier in the production path. It is treated as a prediction, not as structural truth, and is never overwritten by the note generation step.
- **T5** owns only `linguistic_notes`—the user-facing wording. It receives the blueprint text (derived from grammar classes) and the contract context as input; it may only produce note text. It is explicitly forbidden from changing any classifier-owned or structure-owned field.

This ownership matrix is the reason the system is called *contract-safe*: the contract is not a free-form generation output but a structured record whose fields are owned by specific, accountable sources.

Three-Tier CEFR Note Design

A deliberate three-tier design governs how note templates are treated at different proficiency levels:

- **Elementary (A1–A2)**: fixed passthrough templates. Notes describe simple, unambiguous rules that do not vary with the surface content of the matched sentence. They serve as a lower-bound baseline in evaluation.
- **Intermediate (B1–B2)**: slot-templated generation. This is the primary generation task, where the same abstract pattern produces a different, grounded note for each matched sentence via contract-driven slot filling.
- **Advanced (C1–C2)**: fixed passthrough templates. These notes describe complex rules at the construction level. They are likewise invariant to surface content and act as an upper-bound comparison baseline.

By treating the boundary tiers as fixed, the project creates two natural comparison groups that bracket the range achievable by the generation system. Concrete examples from the actual training data are provided in Appendix A.

7.10 Engineering Trade-off

During the work, it became clear that strict dataset cleaning inevitably reduces the amount of material that remains usable for training and evaluation. The raw corpus-based and book-based layers produced thousands and tens of thousands of rows, but after removing repetition, weak formulations, poorly distinguishable explanations, and noisy examples, the working datasets became much smaller. In the final experiments, I was no longer working with the full accumulated mass of material, but with narrower datasets of `3875` and `5016` rows.

This was not the result of an abstract principle adopted in advance. It was the conclusion forced by the intermediate results themselves. As the work progressed, it became clear that simply increasing the number of rows did not improve the system if the new examples mostly repeated explanations that were already present or pushed the model toward a small number of overly frequent answers. In other words, a large dataset did not help by itself if it contained too much repetition and too little genuinely new material.

At the same time, the work still produced several important results. First, a richer structural description of the sentence proved more useful than a shallower input: the system selected the needed explanation more accurately when it saw not only the general shape of the sentence, but also its internal grammatical structure. Second, templated explanations were useful not because they immediately produced the best absolute scores, but because they made the explanation space more ordered and less chaotic. Third, the shift from free text generation to the selection of an explanation type was itself useful. It made the system much easier to evaluate: it became possible to see which explanation it selected, which alternatives it confused, and where its errors began to concentrate.

The limits of these findings are also clear. I can claim that a more structural input helps, that explanation selection as a separate task works, and that the move toward templates was justified. But I cannot yet claim that the current explanation inventory is already broad enough to cover a wide range of grammatical cases reliably. This is especially true for smaller units inside the sentence, where the examples are less stable and less repeatable. In the same way, I cannot yet say with confidence how the system will behave if the number of different explanations becomes much larger: even in the current experiments, increasing the number of alternatives already changed the quality noticeably.

That is the engineering trade-off at this stage. Stricter filtering produced cleaner and more honest results, but at the same time reduced the amount of material on which those results were obtained. For that reason, the work already allows confident conclusions about the direction of the approach, but it does not yet justify treating this part of the system as fully complete.

Chapter 8

Evaluation

8.1 Evaluation Approach

In this work, the evaluation of ELA could not be reduced to one overall metric, because the system is not a single model with one input type and one output type. It consists of several different layers, and an error at each layer means something different. If, for example, the RLE contract is built incorrectly, then later note selection already depends on a distorted representation of the sentence. If the contract is correct but the CEFR prediction is wrong, then the system may choose an explanation at the wrong difficulty level. If both the structure and the level are correct, but the verbal formulation works poorly, the user still receives a weak or badly phrased note. For this reason, a single final score would conceal rather than clarify the real properties of the system.

For that reason, evaluation in the work was separated by layers. Structural correctness had to be checked separately: whether the system parses the sentence into nodes, relations, and grammatical features correctly, and whether the RLE contract remains internally coherent. CEFR prediction quality was also evaluated separately, because it directly affects the level of explanation selected by the system. The task of explanation selection and construction was treated as another separate layer: whether the system can choose an appropriate explanation type, whether it collapses into a few overly frequent answers, and whether a richer structural input gives an advantage over a shallower one. The media pipeline was evaluated separately as well, because for a working product it matters not only how the system analyses text, but also how it behaves when processing user audio, media limits, and related runtime scenarios. Finally, the behaviour of the full user-facing loop also had to be assessed separately, that is, whether the system remains coherent in real execution when the backend, frontend, API, and runtime logic operate together.

That is why this chapter is organised not around one benchmark score, but around several different kinds of evidence. For ELA, it is not enough to report the accuracy of a single model, because that would not answer whether the system is actually interpreting

sentence structure correctly, whether it chooses explanations at the appropriate level, whether it remains stable as the number of alternatives grows, or whether it still works outside an isolated laboratory example. In that sense, layered evaluation is not a matter of methodological ornament, but a direct consequence of the actual architecture of the system.

8.2 Evaluation Method

In this work, I used several different evaluation methods because different parts of `ELA` have to be tested in different ways.

To evaluate sentence structure and internal data, I used checks of `RLE contract` correctness, field schema validity, and consistency of output structures. The purpose here was to determine whether the system was building its internal representation of the sentence correctly and whether that representation remained intact throughout processing.

To evaluate the models, I used standard quality metrics on saved evaluation runs. This applied mainly to `CEFR` prediction, explanation-class selection, and comparisons between different system regimes. These results make it possible to judge whether a richer structural representation of the sentence is helpful, whether explanation selection works as a separate task, and whether the move toward templated explanations is justified.

To evaluate generation quality and intermediate outputs, I used quality-control checks. Their role was not to produce one final score, but to show whether levels, fields, templates, and final notes remained valid after passing through the inference pipeline.

To evaluate the engineering side of the system, I used unit tests and runtime-level checks. These were needed to confirm that the dataset builders, classifier pipeline, API, media processing, validation, and other parts of the system worked not only as separate ideas, but as executable code.

In summary, the work used:

- structural checks for the internal sentence representation;
- model metrics for `CEFR` and explanation selection;
- quality-control checks for inference outputs;
- unit tests and runtime checks for the executable system.

This separation was necessary for `ELA`, because a single metric would not have explained the system properly: it would not have shown separately the quality of the structure, the quality of explanation selection, or the operational state of the system as a whole.

8.3 What the Current Evidence Supports

The current evidence allows me to state that the project has been realised as a working and interconnected system.

This is supported by the testing layer:

- there are currently `194` Python files in `ela_pipeline`;
- there are currently `122` separate test files in `tests/`;
- the key subsystems with the strongest test presence are:
- `classifier`: `53` source files and `46` test files;
- `runtime`: `15` source files and `14` test files;
- `db`: `6` source files and `8` test files;
- `dataset`: `57` source files and `20` test files;
- separate test layers are also present for important applied areas:
- `media`: `7` test files;
- `quality control`: `4`;
- `validator`: `2`;
- `pipeline`: `3`;
- `contract-related`: `3`;
- in addition, there are dedicated tests for `CEFR`, `phonetic`, `translate`, `hil`, `identity`, `tam`, and `synonyms`.

This is supported by the production deployment:

- `ELA` is deployed on a separate Hetzner server;
- the working project path on the server is `/opt/ela`;
- the active branch is `master`;
- the deployed commit on the server is `635a075`;
- the container stack is running under Docker Compose and includes:
- `ela-app`;
- `ela-frontend-1`;
- `ela-postgres`.

This is supported by manual verification of the live instance:

- `https://el-a.uk` responds with `200`;
- the public frontend is available and served correctly;
- the PWA layer and manifest are available;
- a local check of the frontend container through `127.0.0.1:8080` also returns `200`;
- `postgres` is in a working state and accepts connections.

This is supported by the observed system behaviour in the logs:

- the containers have been running for weeks without signs of crash behaviour;
- the recent logs are dominated mainly by external scanning and low-value traffic;
- requests to `/.env`, `phpunit`, `wp-login`, `actuator/health`, and similar paths do not expose sensitive data;
- the system responds to such requests with refusal codes (`404`, `405`, `501`) rather than unstable behaviour.

Therefore, the current evidence allows me to state that:

- the project exists not only as a research idea, but as an executable system;
- its key parts are being tested systematically;
- it is genuinely deployed as a containerised service on Hetzner;
- its main components operate as a connected stack rather than as isolated modules.

8.4 Classifier Path and Controlled Generation Evidence

For the evaluation of the classifier path in this work, not only the final results matter, but also the transition from the early working pipeline to the later, scaled runs. One of the key early checkpoints here is the classifier execution report dated `2026-02-25`. It confirms that by that point there already existed not just a standalone training script, but a connected cycle: an orchestrator run, `KB` building, train-dataset preparation, classifier training, metadata generation, quality-cycle execution, runtime sentence-contract validation, and controlled note rewriting. In other words, even at that stage the classifier path already existed as a real working pipeline rather than as an isolated experiment.

At the same time, that same checkpoint also records an important limitation. In this early baseline freeze, the training set was still very small: `train=4`, `dev=2`. Therefore, this report supports a strong conclusion about the existence and operational coherence

of the pipeline itself, but it does not support strong conclusions about classifier quality in any broader sense. This stage confirms that the pipeline had already been assembled and could complete a connected run, but it does not confirm that it was yet sufficiently scaled for serious modelling claims.

For that reason, this early checkpoint has to be read together with the later classifier artefacts. Later, the classifier path did not remain at the level of a tiny baseline freeze, but was extended into a full-ladder tabular run on a much larger dataset. In the saved `tabular_joint_profile_full_ladder_eval_2026-03-05.json`, the run already contains `8211` training rows, `1182` development rows, and `1196` test rows. By that stage, the classifier path had not merely been preserved, but scaled by an order of magnitude compared with the early phase. This matters because it shows the actual evolution of the work: from a small proof that the pipeline could function to a substantially larger training and evaluation run.

It is equally important that as the project developed, the classifier path did not merely grow in size, but was compared against alternatives. The saved comparative artefacts show that the tabular/XGBoost branch was, in practical terms, stronger than one of the full-ladder `DeBERTa` branches. Therefore, the classifier path in this work did not remain a "proof of concept" by inertia. It was genuinely subjected to scaling, comparison, and stress-testing, and later architectural decisions were made on the basis of those results.

8.5 Concrete Tabular Evaluation Snapshot

The previous section showed that the classifier path in the work developed from a minimal working run into a much larger and more substantial form. However, that transition by itself does not yet show what state the system had reached in numerical terms. For that reason, it is useful to consider one concrete evaluation checkpoint that provides not only a general sense of development, but also explicit dataset sizes and metrics.

In this work, that checkpoint is the tabular evaluation snapshot dated `2026-03-05`. It is important because it records not an early validation stage, but a more mature run on a large dataset. In this run, the model predicted not only `CEFR`, but two characteristics of the sentence at once: the difficulty level and the primary grammatical class. In other words, the system had to answer two questions simultaneously: how difficult the construction was and what grammatical type it belonged to.

Table 8.1: Selected values from the 2026-03-05 tabular joint-profile evaluation artifact

Metric or count	Value
Training rows	8211
Development rows	1182
Test rows	1196
Joint development accuracy	0.9797
Joint test accuracy	0.9849
Grammar development macro-F1	0.8088
Grammar test macro-F1	0.8231
Runtime smoke check	pass

These figures need to be read directly. Here, `joint accuracy` means that the model had to predict both characteristics correctly at the same time: the `CEFR` level and the grammatical class. In the same run, the separate `CEFR` scores reach `1.0`, while the more difficult part remains grammatical classification, where `macro-F1` stays at about `0.81-0.82`. This shows that in this case difficulty-level prediction was no longer the main challenge, while distinguishing between grammatical types remained the harder task.

This snapshot therefore makes the conclusion of the previous section more concrete. There, the point was that the classifier path had not only been assembled, but had also been developed substantially. Here, it becomes clear what that development meant at one later stage: a large training run in which the system was already solving a more demanding task than at the beginning and producing a measurable result for it.

At the same time, such metrics cannot be detached from the conditions under which they were obtained. They belong to a specific run, a specific dataset, and a specific joint-classification task. For that reason, this snapshot should be used in the thesis not as a universal number for the whole system, but as a measurement point showing the state that the classifier path had reached at this stage of the work.

8.6 Comparative Evidence and Intermediate Results

After the concrete snapshot in the previous section, it is important to consider not only the stronger checkpoints, but also the comparisons between different branches of the work and the intermediate results that turned out to be weak. This matters because the architectural decisions in the work were not made according to a fixed plan in advance, but in response to the outcomes of actual runs.

One of the clearest examples is the comparison between the tabular full-ladder path (that is, the branch in which `XGBoost` predicted the class not from free text, but from

explicitly defined features such as syntactic signatures, grammar classes, tense-aspect-mood characteristics, node type, node role in the RLE tree, the selected template, and other contract fields) and the DeBERTa branch from March 2026. This comparison showed that the tabular branch had become the more practical option for the working system configuration. In one of the saved DeBERTa CEFR-only runs, accuracy was only 0.1612 and macro-F1 was only 0.0463. At the same time, the model effectively collapsed into a single class and mapped almost all predictions to C1. As a result, the heavier classifier branch did not produce the stronger practical outcome at that stage, and this directly explains the move toward the tabular route as the more workable option.

A similar role was played by the intermediate results in the note-generation branch. Early and intermediate T5 runs showed that simply training a model on notation-like targets did not automatically produce useful pedagogical output. Several evaluation reports showed very weak exact match, in many cases simply 0.0. This applied to part of the refresh runs, the template-id experiments, and several early T5 note models. In other words, generation against a target that looked like an explanation was not, by itself, enough to solve the task or to produce a result strong enough for practical use.

A direct conclusion followed from this in the work. The later architecture was chosen as narrower but more controllable: the structure of the sentence was fixed first in the RLE contract, then the level and the explanation type were determined separately, after that the system moved to a template, and only at the final stage did it produce the verbal formulation of the note. In other words, the weak results of the comparative and intermediate runs showed which solutions were not providing the required stability, and this is what led to the shift from more open-ended generative schemes to a more tightly constrained pipeline.

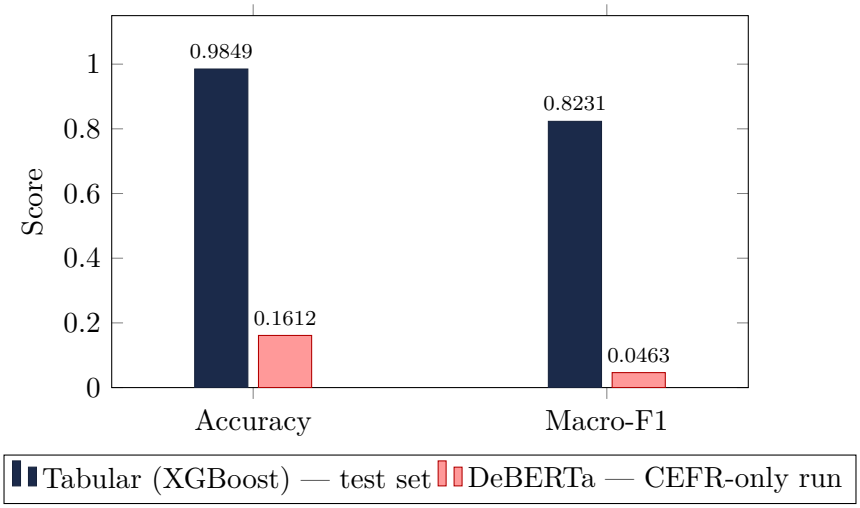

Figure 8.1: Classifier comparison: tabular XGBoost (2026-03-05 test set) versus DeBERTa CEFR-only run. The DeBERTa branch collapsed predictions to a single class (C1), while the tabular path achieved a joint test accuracy of 0.9849.

8.7 What the Current Evidence Does Not Fully Support

At the same time, the current evidence still does not justify very strong claims about the completeness or final maturity of the system. The main reason is not the absence of working components, but the narrowness of the final working datasets on which the main later results were obtained, and the high cost of producing those datasets.

As shown above, the final working datasets in the note-selection experiments contained `3875` and `5016` rows. These are no longer tiny trial samples, but they are also not broad enough to support unrestricted claims about complete coverage of the explanation space. These datasets were not simple slices taken directly from a large raw corpus. They were the result of a long selection process: removing repetition, filtering weak formulations, limiting overly frequent targets, balancing, templating, parameterisation, and checking structural suitability. In other words, the final datasets were obtained at a high cost, and for that reason they remained relatively narrow compared with the original volume of material.

This limits the strength of the conclusions in a concrete way:

- I can claim that the chosen direction works on these disciplined working datasets;
- I can claim that a more structural input and a more controlled explanation-selection process bring a practical benefit;
- I can claim that the move toward templates and controlled rendering was justified;
- but I cannot yet claim that this is already enough for broad and even coverage of the whole note space.

The same applies to the phrase-level branch. As shown above, it remained noticeably weaker than the sentence-level layer. This means that even with strong later checkpoints, I cannot claim that the whole system is already supported equally well at every level. By this stage, the sentence-level branch appears more mature, while phrase-level coverage remains narrower and less stable.

For that reason, the most accurate conclusion here is straightforward. The current evidence confirms that the project has been realised technically and that the chosen architecture produces a working result. But it does not yet confirm that the present scale and composition of the datasets are already sufficient for broad claims about the completeness, balance, and final maturity of the whole note-generation layer.

8.8 Representative Findings from Review

The review showed that the project had already been assembled not as a set of isolated modules, but as a system of connected parts. By this stage, the strongest interaction existed between sentence structure analysis, difficulty-level prediction, explanation-type selection, and final note construction. This is one of the strongest sides of `ELA`: the system does not simply try to write the final text immediately, but first builds an internal description of the sentence, then determines how difficult the construction is and what exactly needs to be explained, after that selects an appropriate form of explanation, and only at the final stage moves to verbal wording. As a result, the main decision about the content of the explanation no longer remains entirely inside free generation.

The preparation of data, the training of models, and the later evaluation of results were also strongly connected. In the project, the datasets did not exist separately from the models: the way the data were cleaned, balanced, templated, and reduced into a more disciplined set directly affected what tasks could later be given to the models and what results could be obtained from them. This is a strength of the work because there is a direct connection here between the structure of the data and the structure of the system itself, rather than two layers that happen to evolve side by side.

Another strength of the project is that its internal processing is already connected to a real user-facing loop. `ELA` exists not only as a set of offline experiments, but as a deployed system in which the frontend, backend, database, and runtime services operate together. This matters because it allows the project to be treated as a realised software system rather than only as a research pipeline.

The review also showed weaker points in the interaction between the parts. Most importantly, the work with whole sentences is noticeably more mature at this stage than the work with smaller internal fragments inside them. For full-sentence explanations, there is already stronger template coverage, a clearer explanation-selection path, and more stable later results. For the phrase level, the same chain remains narrower and weaker. In other words, the system is more fully developed where it explains the sentence as a whole than where it must explain smaller internal structures in a stable way.

The interactions inside the multimodal path also remain weaker. By this stage, the text path already appears more direct and disciplined: a sentence enters the system, is analysed, receives a difficulty level, an explanation type, and a final note in a relatively clear sequence. The media path is longer and more fragile: it must first go through ingestion, ASR, segmentation, metadata filtering, and sentence extraction before it can even enter the main linguistic pipeline. For that reason, the media branch depends more heavily on the quality of its intermediate stages and is less tolerant of errors at the input side.

Finally, the review showed one more important imbalance. The project already con-

nects structural analysis, explanation selection, and controlled note construction quite well, but the connection between this strong architecture and the completeness of the overall explanation space remains less fully supported. In other words, the working scheme of the system already appears mature, but the breadth of the explanation inventory that has actually been confirmed still lags behind the maturity of the scheme itself.

8.9 Overall Assessment

Overall, the project can be evaluated as a realised research-engineering system.

First, it is broad enough in scope. The work includes not only models and datasets, but also the full applied loop: sentence-structure analysis, explanation selection, controlled note generation, the runtime layer, the frontend, the backend, the database, and production deployment. For that reason, `ELA` can be treated not as an isolated research experiment, but as a full-stack system with real interaction between its parts.

Second, the project is deep enough in its internal content. At this stage, it is possible to discuss not only the general idea, but also concrete architectural and modelling decisions: why the `RLE contract` was needed, why free generation turned out weaker than a more controlled scheme, how templates changed the task, how the classifier path evolved, and why the later architecture became narrower and more controllable. In other words, the project provides material not only for demonstrating a result, but also for substantive technical analysis.

Third, the work remains honest in its results. A careful review shows not only what has already been built, but also the gap between implemented ambition and full production hardening. This does not weaken the project as a research effort. On the contrary, it makes it more convincing: the strengths are supported by facts, while the limitations are neither hidden nor disguised behind overly general claims.

Finally, the project preserves enough intermediate artefacts to make it possible to trace why the later decisions were taken in the form they were. The history of the datasets, the comparative runs, the quality-control materials, and the evaluation checkpoints all show that the architecture did not change arbitrarily, but in response to real intermediate results. For that reason, `ELA` can be evaluated not only by its final state, but also by the consistency with which it accumulated and interpreted engineering evidence along the way.

8.10 End-to-End Integration Perspective

At this stage, it is already clear that the monolithic architecture used in the project produced a strong result during the phase of rapid research, pipeline assembly, and hypothesis

testing, but is beginning to exhaust its usefulness as a long-term basis for further development. Within a single code contour, the system already brings together structural parsing, CEFR classification, explanation selection, controlled note construction, media processing, runtime services, frontend integration, persistence, and deployment logic. At the earlier stage, this was useful because it allowed new ideas to be connected quickly and tested on real material. As the project has grown, however, the same architecture has begun to make scaling, change isolation, and stable development of separate parts more difficult.

The main problem is that different parts of ELA now move at different speeds and have different technical natures. Sentence-level linguistic analysis, note-selection models, media workflows, frontend state, and backend orchestration place different demands on data, latency, update cycles, and responsibility boundaries. Inside a monolith, these differences can still be held together, but further growth will make that arrangement increasingly heavy. Changes in one area begin to affect others more strongly, and the development of the system as a whole becomes less transparent.

For that reason, the next logical step for stable development and future scaling is a transition toward a data-driven microservice architecture with a hexagonal organisation. The point of such a transition is not simply to split the monolith into smaller parts, but to establish stable boundaries around data and responsibility. In such a structure, separate services or domain contours would be defined for linguistic analysis, classifier inference, note rendering, media processing, storage, and delivery, while their interaction would be organised through explicit contracts, events, payload schemas, and adapter layers. This would make it possible to develop the internal logic of each part without constant pressure from the rest of the system.

This matters especially for ELA because the project has already moved beyond the stage where the main challenge was simply building separate working components. The challenge is now shifting toward how those components can continue to evolve together in a stable way. For that reason, the end-to-end integration perspective here should be understood as an architectural transition: from a research monolith that was well suited to the earlier phase, toward a more data-driven and hexagonal system capable of stable growth, independent subsystem updates, and more mature product-scale deployment.

8.11 Objectives-to-Evidence View

In the Objectives section of the present thesis, four main goals of the implementation phase were defined. For that reason, at the end of the evaluation chapter it is important to show not simply a collection of results, but how those results make it possible to treat each of these goals as fulfilled. The table below aligns the goals of the present thesis with the outcomes that support such a conclusion.

Table 8.2: Alignment between the goals of the present thesis and the results that support their fulfilment

Objective	Intended outcome	What shows that the goal was achieved
O1	Construct a multimodal pipeline that takes in text, audio, and video and converts them into structured sentence-level representations	the system includes a text path, a media path, the sentence-contract pipeline, runtime services, the API layer, and a deployment contour, so the input material does in fact pass through to a structured sentence-level result
O2	Formalize a hierarchical contract capable of carrying grammatical, pedagogical, and bilingual metadata in a stable and validated form	the work implements the `RLE contract`, validator logic, schema definitions, sample contracts, and contract-safe processing rules, so this structure is not only described, but actually used as a working foundation of the system
O3	Provide a learner-facing application that exposes linguistic structure through an interactive visualizer, export flow, and media-oriented workflow	the project includes the frontend, visualizer-related components, export logic, runtime endpoints, and a deployed PWA/frontend contour, so the result has been carried through to a user-facing application rather than remaining only a backend pipeline
O4	Explore whether abstract structural templates support stronger note-generation generalization than raw lexical notation	the work moved from raw lexical generation toward template-based and placeholder-based paths, carried out comparative runs, built controlled-rendering logic, and produced later note-selection datasets, so this direction was not only proposed, but actually tested experimentally

This table is needed in order to return the evaluation chapter to the frame that was defined earlier in the `Objectives` section. It shows that the goals of the present thesis did not remain declarative. The first goal is supported by the implemented multimodal pipeline. The second is supported by the fact that the contract exists as a working

and validated structure. The third is supported by the fact that the system has been brought through to a learner-facing application. The fourth is supported by the fact that structural abstraction in note generation was actually tested through data, models, and the later controlled architecture.

The final conclusion here is straightforward: with respect to the goals that were explicitly defined in the `Objectives` section of the present thesis, the implementation phase can be regarded as completed. This is supported not by one overall metric, but by a combination of results: architecture, executable code, the runtime contour, deployment, dataset engineering, and experiments on note generation.

Chapter 9

Discussion, Limitations, and Future Work

9.1 Discussion

The results of the work broadly support the central direction of the thesis: authentic language material can indeed be made more accessible not through simplification, but through structured explanation. By this stage, the strongest part of `ELA` is the connection between the hierarchical representation of the sentence, the rigid structural foundation of the system, and the learner-facing visualizer. This is the most coherent part of the project because it brings together linguistic analysis, the internal contract, and the user-facing way of working with the result into one clear scheme.

One of the most important findings of the work is that, for pedagogical note generation, representational form mattered more than simply making the model more complex. The clearest progress was achieved not when the task remained free text generation, but when it was translated into a more structured and constrained form. The move toward templates, placeholders, and controlled rendering produced a more useful result than trying to force the model to reproduce a large number of surface formulations directly. In other words, the system became stronger when the model stopped trying to memorise many separate textual variants and instead worked with a more stable structural scheme of explanation.

This is also the point at which the research and engineering sides of the thesis meet most clearly. The same principles that led to a stricter `RLE contract`, validator logic, and a more controlled runtime pipeline also led to the restructuring of the note-generation path. In both cases, the work moved in the same direction: from less constrained and less transparent solutions toward more explicit, structured, and verifiable forms. The more hidden variation was converted into explicit structure, the more stable the system became.

9.2 From LLM Pattern Learning to a General Scientific Method: A Pattern-Sequence Hypothesis

The hypothesis formalised in this section did not arise as a pre-planned theoretical claim. It grew out of practical observations during the implementation of `ELA` and was later noticed again in a second, physically unrelated domain. It is this reappearance of the same core idea in two different areas—language pedagogy and meteorological forecasting—that allows it to be treated not merely as a local engineering conclusion, but as a candidate for a more general principle.

Origin in the ELA Pipeline Failure

The first encounter with this principle came through a practical failure. In the early `T5` note-generation branches, the training signal preserved too much surface lexical form. As a result, the model memorised individual examples instead of learning transferable pedagogical patterns. This became especially clear in the template-ID experiment of February `2026`: after target-space balancing, the effective inventory collapsed to only `10` unique template identifiers, while the fallback rate on the small regression probe reached `1.0`. In other words, the model learned nothing useful because surface variation dominated the hidden structure, and the model memorised that variation instead of the structure beneath it.

The turning point came when raw notation was replaced with structural templates containing placeholders. After that, the behaviour of the system changed qualitatively: the model began to generalise better across different lexical realisations while preserving grammatical interpretation. This was the first clear indication that the transferable signal was not in the surface form, but in the structural pattern.

A Historical Precedent: Mendeleev's Grammar

The clearest analogy for this observation appears long before machine learning. Mendeleev did not simply list chemical elements. He showed that elements, when arranged according to their internal properties, form repeating patterns, and that these patterns have predictive force. When a gap appeared in the sequence, this did not invalidate the system. On the contrary, it made it possible to infer the existence of a missing element and to predict its properties from its place in the overall structure.

If the chemical content is removed, a more general method remains: identify the minimal analytical unit, describe its properties, study the regularities of its sequential arrangement, and then use those regularities for prediction. In that sense, the Periodic Table is not just a list of elements, but a structure of transitions between them. This

is what makes the analogy important for `ELA`: the system becomes stronger not when it memorises more surface examples, but when it learns to recognise and use a stable sequence of structural patterns.

Language as the First Domain

In ELA, the analytical unit is the Recursive Linguistic Element node. Each sentence, phrase, and word node is described not by its specific lexical content but by its structural type, grammatical role, tense-aspect-mood classification, and position in the hierarchy. The placeholder-based training signal then asks the model to predict which structural pattern applies to a new example—which note template, which grammatical regime—rather than to memorise which words appeared in training. This is the open phase of the duality: many contextual examples, each with different surface vocabulary, carve a stable internal pattern space. In the closed inference phase, the frozen pattern then imposes structure on whatever new context arrives.

The same logic can also be seen in the way a human being learns a foreign language. At the early stage, the learning system remains open: the learner encounters a large number of new contexts, words, constructions, and usage situations. At that point, it is the context that gradually forms the internal patterns. In other words, the learner does not yet possess a stable structure in advance, but extracts it from many particular examples.

As learning progresses, however, the system begins to close in a different sense: the patterns that have already been acquired start to constrain and organise the interpretation of new material. Once the learner recognises, for example, a conditional structure, a passive form, or a modal-perfect pattern, a new sentence is no longer perceived as a chaotic stream of words. On the contrary, the known structure already provides a frame for what can be realised there and how it should be interpreted. In other words, at the early stage, context builds the pattern, while at the more mature stage, the learned pattern structures the acceptable context of interpretation.

This is why the analogy with language learning is especially strong here. A person does not acquire a language as an endless list of independent surface realizations. It is acquired through the gradual extraction of stable structural patterns, and those patterns later begin to govern the understanding of new examples. In that sense, foreign-language learning itself already demonstrates the transition from an open system, in which many contexts form the pattern space, to a more closed system, in which acquired patterns organise later perception and prediction. This statement is formulated explicitly in the core hypothesis:

> Effective training of an LLM is achieved when both the input and the output are represented as patterns with placeholders, and variability is realised not as

a separate collection of individual cases, but as a structured system of filling those placeholders within the pattern.

This gives rise to the hypothesis that, in open systems, context forms patterns, whereas in closed systems, stable patterns structure the permissible context of their realisation.

In turn, this suggests that the complexity of natural systems is determined not by an infinite multitude of variants, but by a limited set of stable patterns through which those variants become possible, reproducible, and interpretable.

Weather Forecasting as the Second Domain

The most direct external evidence for the generality of this approach comes from parallel work on single-station meteorological forecasting [15] conducted at Knock Airport, Ireland, using Met Éireann historical hourly data covering the period 2020–2026. In that system, raw sensor measurements are not fed directly to a model. Instead, the time series is first segmented into analytical intervals, each described by one of a small library of ODE-derived equation types: `constant`, `linear`, `exponential`, `harmonic`, and `damped_harmonic` for temperature, and a three-type vocabulary for pressure and wind speed. The choice of equation type and its fitted parameters constitute the vocabulary of meteorological regimes in precisely the same sense that RLE node types constitute the vocabulary of grammatical regimes in ELA.

A transformer encoder is then trained to predict the *sequence* of future ODE segment descriptors from a window of past ones. The model receives no atmospheric physics, no spatial observations, and no numerical weather prediction output. It learns only the sequential grammar of analytical meteorological units from six years of a single station's history. On a seven-day evaluation period in February 2026, the ensemble achieves a pressure mean absolute error of 0.89 hPa at the 12-hour horizon, a 34% improvement over naive persistence (1.35 hPa), using no domain physics whatsoever. This is not a claim of superiority over operational numerical weather prediction systems; it is a proof of concept that the *grammar of physical regimes is learnable from sequential analytical units alone*, exactly as the grammar of linguistic regimes proved learnable in ELA.

The structural parallel between the two domains is summarised in Table 9.1.

Table 9.1: Structural parallel between the ELA and weather forecasting pattern-sequence approaches

Concept	ELA (language domain)	Weather forecasting domain
Analytical unit	RLE node (sentence, phrase, word)	ODE segment (constant, linear, exponential, harmonic)
Unit descriptor	type, TAM, grammatical role, CEFR level	equation type, fitted parameters, duration
Sequence model	transformer over RLE node sequences	transformer over ODE segment sequences
Training signal	structural templates with placeholders	segment descriptor vectors
Predicted output	next structural pattern and its template slots	next segment type and fitted parameters
Decoded output	learner-facing pedagogical note	hourly meteorological forecast values

Formulation of the General Method

These two cases make it possible to extract a three-step method that may apply to any sufficiently regular natural or artificial process.

1. **Decomposition.** First, it is necessary to identify the minimal analytical unit appropriate to the domain. This unit must be expressive enough to capture the essential structure of a regime, compact enough to form a finite vocabulary, and grounded in domain knowledge rather than derived only from statistical clustering. In language, this unit is the grammatical node with its `TAM` and role attributes. In meteorology, it is the `ODE` segment type together with its fitted parameters.

2. **Classification and sequence.** The next step is to recognise that real instances of the process form sequences of such units. The units must then be classified, their transitions recorded, and the main object of analysis treated not as raw surface values, but as the sequence of analytical units itself. The Periodic Table is the classical example of this step in chemistry: once elements are classified by their properties, the structure of their arrangement becomes visible and predictive.

3. **Learning the grammar and prediction.** After that, a sequence model is applied to the stream of classified units. Because the model operates not on raw measurements but on analytical descriptors, it learns not surface variation but the structural regularities of transitions between regimes, that is, their grammar. The prediction can then be unfolded back into surface form through the analytical units

themselves, preserving interpretability at every step.

A New Methodology of Scientific Inquiry

The pattern-sequence approach is worth situating not only as a machine-learning technique but as a methodology of scientific inquiry in its own right—one that became practically realisable only with the arrival of modern sequence models. Prior to the transformer era, identifying the analytical units of a domain and studying their sequential grammar was work reserved for domain experts investing years of manual analysis. The Periodic Table took Mendeleev a lifetime; the grammatical rules of natural language occupied generations of linguists; the ODE taxonomy of meteorological regimes exists only as informal folklore among forecasters. What the AI revolution makes possible is the *automation of the grammar-learning step*: once a domain can be expressed as a sequence of interpretable analytical units, a transformer can extract the sequential regularities from historical data at a scale no human analyst could match.

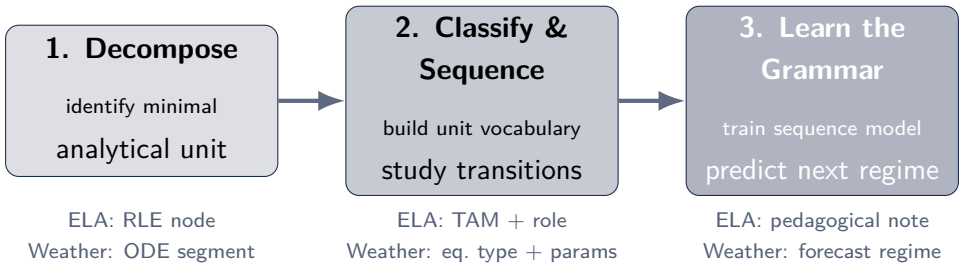

Figure 9.1: The pattern-sequence three-step method: decompose the domain signal into analytical units, classify and observe their sequential grammar, then train a sequence model to predict the next unit. Domain-specific instantiations are shown below each step.

This positions the three-step method—decompose, classify, learn the grammar— as a candidate general framework for data-driven scientific understanding, distinct from both classical hypothesis-driven science and from the black-box statistical learning that characterises much of modern deep learning. It is interpretable because the units are analytically defined; it is powerful because the grammar is learned from data; and it is generalisable because the same architectural pattern—segment, describe, sequence-model— applies wherever recurring analytical regimes can be identified. The two domains examined in this thesis, natural language and meteorology, may therefore be understood not as isolated examples, but as early evidence of a broader shift toward extracting structured knowledge from complex systems through analytical units and their sequential grammar.

Scope and Limitations of the Hypothesis

The hypothesis should be understood as an empirical claim derived from two implemented cases, not as a formal theorem. Several important qualifications apply.

First, the quality of the analytical decomposition is a decisive factor. If the vocabulary of units is too coarse, important distinctions collapse; if it is too fine, the model faces a large and sparsely populated unit space that is difficult to learn from. In both ELA and the weather project, vocabulary size was determined through domain knowledge and iterative refinement rather than by automated search, and that design investment is not free.

Second, the generalization gains observed in both domains are bounded by evaluation scope. ELA's placeholder-based note generation was evaluated on a controlled annotation set; the weather forecasting result covers one week at one station during a challenging frontal passage. Neither result is sufficient to claim general superiority over all alternative approaches.

Third, the claim is not that all complexity reduces to patterns in a mystical sense. The hypothesis is more practical: in domains where analytical decomposition is feasible, learning the grammar of analytical unit sequences is a more efficient and more interpretable strategy than learning directly from raw surface data. Whether that efficiency advantage survives in richer, noisier, or higher-dimensional domains is an open research question.

Pattern-Sequence Hypothesis, representing a more general formulation of the context–pattern duality applied in this work, together with the analogy to Mendeleev, constitutes the part of the research contribution of this thesis that extends beyond ELA's engineering work. It represents a design philosophy tested in two domains and presented here as a candidate framework for further systematic investigation.

9.3 Limitations

If the results of this chapter are taken as a whole, the main limitation appears in the same form across both domains considered here. In both `ELA` and meteorological forecasting, the architectural and research idea has already shown that it can work, but its further development runs into the same bottleneck: the quality, scale, and cost of preparing analytically usable data.

In the language domain, this means that it is no longer enough simply to have working datasets of a few thousand rows. Stronger generalisations require substantially larger datasets that are at the same time carefully cleaned and structurally usable. They must not merely contain text, but preserve a correct contract-compatible structure, a distinguishable grammar space, suitability for template abstraction, and sufficient pedagogical

usefulness. Otherwise, growth in volume quickly turns not into stronger signal, but into an accumulation of repetition, noise, and weak explanatory cases.

In the meteorological domain, the limitation has a similar form. The approach based on analytical units and sequence modelling produced a convincing result for one station over a limited time span, but this is still not enough for broader claims. To move from proof of concept to stronger conclusions, a much wider body of observations is needed: more stations, more regimes, more weather situations, and a longer history over which the stability of this representational advantage can be tested.

This produces the main generalisation across the two domains. The bottleneck is not primarily at the level of the general idea, because the idea has already produced meaningful results in two different areas. The bottleneck lies at the level of how much well-decomposed, structurally disciplined material can be assembled for its further development. In other words, the central difficulty is no longer whether such a system can be built at all, but whether it can be supplied with a sufficiently rich and sufficiently clean space of analytical units.

From this follows a broader conclusion about the Pattern-Sequence Hypothesis itself. It already appears strong as a research framework and as a working method, but not yet as a fully established general law. Its next stage does not require a new formulation so much as broader testing on larger and better-covered datasets across different domains. For that reason, the main limitation of this chapter is best described in the following way: the method has already shown explanatory and practical strength, but its further confirmation now depends primarily on the scale and quality of structurally usable data.

9.4 Future Work

The next stage of the project should not be centred on adding isolated new features, but on moving the whole system into a more mature phase of development. By this point, `ELA` already has a working architectural foundation, a research line, and a set of implemented components. For that reason, future work should be organised around three broader directions: expanding the layer of high-quality data, restructuring the architecture of the system, and carrying out broader validation of the result.

The first and most important direction is the growth of a higher-quality dataset layer. The results of this thesis already show that data are becoming the main constraint on further development, both for the note-generation line and for the more general Pattern-Sequence Hypothesis. In the language domain, this means assembling a substantially larger but still disciplined corpus: not simply many rows, but material that preserves contract-compatible structure, a distinguishable grammar space, suitability for template abstraction, and pedagogical usefulness. At the next stage, the project needs a high-quality body of at least about `25,000` rows that is usable for more stable work with both

sentence-level and phrase-level note space. Without that, further model improvement will produce progressively weaker returns.

The second direction is architectural restructuring. As discussed above, the monolithic architecture worked well during the phase of rapid research and early integration, but it is beginning to exhaust its usefulness as a basis for stable growth. For that reason, the next development phase should involve a move toward a more data-driven microservice architecture with a hexagonal organisation. In practical terms, this means clearer separation between the contours of linguistic analysis, classifier inference, note rendering, media processing, storage, and delivery, together with explicit contracts between them. This restructuring matters not for abstract engineering elegance, but so that different parts of the project can evolve at different speeds without repeatedly destabilising one another.

The third direction is the expansion of the evidence base. At the current stage, the thesis already shows that the chosen architectural and research line works, but further maturity requires broader validation. For `ELA`, this means larger and better-covered experiments in the language domain, further testing of the Pattern-Sequence Hypothesis on new kinds of data, and eventually movement beyond purely technical evaluation toward a user-study layer. In other words, the next serious stage of the work should answer not only whether such an explainable system can be built, but also how far it actually improves learner comprehension, retention, and usable educational value under more realistic conditions.

Reduced to its essentials, the future work of the project is no longer defined by the search for a general idea. By this point, the idea, the architectural framework, and the research direction are already in place. The next phase depends primarily on three things: larger high-quality data, a more mature architecture, and broader empirical validation. That is the natural continuation of this thesis.

Chapter 10

Conclusion

This thesis has shown that ELA was able to move from its earlier research formulation into an implemented system. The project no longer exists only as an idea of explainable support for authentic English input. By this stage, it has already taken shape as a full-stack system built around the `RLE contract`, a rules-first linguistic pipeline, multimodal processing, and a learner-facing frontend.

The main technical conclusion of the work is that explainability requires not only models, but architecture above all. For ELA, what proved decisive was not individual successful model outputs by themselves, but the existence of a contract-safe structure, validator logic, runtime boundaries, and a stable way of connecting linguistic analysis to learner-facing explanation. In other words, the system became substantively stronger when its explainability began to be guaranteed not by the promise of a "smart model," but by the structure of the architecture itself.

The research conclusion is no less important. The work showed that, in note generation, the decisive factor is not so much greater model complexity as the level of structural abstraction in the representation of the task. The move from raw lexical notation to templates, placeholders, and controlled rendering produced a more stable result than attempts to generate a large number of surface formulations directly. It is from this result that the broader research line of the thesis emerged: from context-pattern duality to the Pattern-Sequence Hypothesis as a wider formulation of the idea that transferable signal lies not in the surface itself, but in the stable sequence of structural units.

At the same time, the work does not conceal its own limits. ELA cannot yet be treated as a fully completed product, and the Pattern-Sequence Hypothesis cannot yet be treated as a definitively established general law. In both cases, further development now depends above all on larger high-quality data, a more mature architecture, and broader testing under different conditions. But this is precisely what makes the current result valuable: the thesis records not an abstract promise, but a system that has already been built, tested, and made capable of further development.

In that sense, the continuation of the earlier research thesis can be regarded as suc-

cessful. The original pedagogical idea was not lost in the transition to implementation. On the contrary, it was translated into contracts, validators, services, datasets, runtime logic, experiments, and real limitations. That transition from concept to accountable system is the main result of this work.

Appendix A

Pipeline Template Examples

This appendix provides concrete examples drawn from the actual training data generated by the closed pipeline described in Section 7.9. As explained there, the pipeline uses a three-tier design:

- **Elementary (A1–A2)**: fixed passthrough templates, used as a lower-bound comparison baseline;
- **Intermediate (B1–B2)**: slot-templated generation, the primary modelling target;
- **Advanced (C1–C2)**: fixed passthrough templates, used as an upper-bound comparison baseline.

For each example the table shows the source phrase or sentence, the grammatical topic as labelled in the source book, the note template (with any `{{SLOT}}` placeholders shown explicitly for intermediate examples), and the final rendered note that would be displayed to the learner.

Elementary Level (A1–A2): Passthrough Notes

At the elementary tier the note text is typically short, unambiguous, and directly applicable regardless of the specific sentence. These notes are stored as passthrough templates with no slots.

Table A.1: Elementary-level passthrough note examples from the training corpus

Source phrase / sentence	**Topic**	**Note (passthrough template)**
I'm going to go downtown tomorrow.	be going to future	Be going to is used to talk about a future plan or intention.
Ann isn't going to study tonight.	be going to negative	In negative be going to clauses, not follows be.
Are you going to come to class tomorrow?	be going to yes-no question	A yes-no question with be going to is formed by placing be before the subject.
What time are you going to eat dinner tonight?	be going to wh-question	A wh-question with be going to begins with the wh-expression and keeps be before the subject.
There is a book on my desk.	existential there basic	There is introduces the existence of a singular thing in a place or situation.

Intermediate Level (B1–B2): Slot-Templated Notes

At the intermediate tier many note patterns contain references to specific grammatical elements of the target phrase. These are stored as slot templates; at projection time the slots are filled from the linguistic contract of the matched corpus instance, producing a grounded note for each new context.

Table A.2: Intermediate-level slot-templated note examples from the training corpus

Target phrase	**Topic**	**Note template (with slots)**	**Rendered note (slots filled)**
that you hit	relative clause antecedent	This relative clause gives more information about the noun "{{HEAD_NOUN}}".	This relative clause gives more information about the noun "car".
that I met	object relative clause	This relative clause gives more information about the noun "{{HEAD_NOUN}}".	This relative clause gives more information about the noun "woman".
that you talked to	stranded preposition relative clause	This relative clause gives more information about the noun "{{HEAD_NOUN}}".	This relative clause gives more information about the noun "person".
for my birthday	prepositional phrase	Here, "{{OBJECT_NP}}" functions as the object of the preposition "{{PREPOSITION}}".	Here, "my birthday" functions as the object of the preposition "for".
with a hammer	prepositional phrase	This prepositional phrase contains the preposition "{{PREPOSITION}}" and the object "{{OBJECT_NP}}".	This prepositional phrase contains the preposition "with" and the object "a hammer".
who I was talking about	relative clause	This relative clause gives more information about the noun "{{HEAD_NOUN}}".	This relative clause gives more information about the noun "woman".

The slot mechanism makes the difference across the three tiers concrete. Elementary and advanced notes are self-contained pedagogical rules whose text does not change with the sentence: they are passthrough templates. Intermediate notes are abstract patterns

that become specific only when instantiated against a real contract: they are slotted templates. All three forms pass through the same closed pipeline; only the template kind—passthrough versus slotted—differs. The two fixed tiers serve as natural baselines that bracket the quality range achievable by the generation system.

Bibliography

[1] A. Gilmore, "Authentic materials and authenticity in foreign language learning," *Language Teaching*, vol. 40, no. 2, pp. 97–118, 2007.

[2] W. Guariento and J. Morley, "Text and task authenticity in the EFL classroom," *ELT Journal*, vol. 55, no. 4, pp. 347–353, 2001.

[3] S. A. Berardo, "The use of authentic materials in the teaching of reading," *The Reading Matrix*, vol. 6, no. 2, pp. 60–69, 2006.

[4] G. Levy, *Computer-Assisted Language Learning: Context and Conceptualization*. Oxford, UK: Clarendon Press, 1997.

[5] C. A. Chapelle, *Computer Applications in Second Language Acquisition*. Cambridge, UK: Cambridge University Press, 2001.

[6] M. Honnibal and I. Montani, "spaCy 3: Industrial-strength Natural Language Processing in Python," 2023.

[7] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[8] A. Radford et al., "Robust speech recognition via large-scale weak supervision," in *Proceedings of ICML*, 2023.

[9] A. Fan et al., "Beyond English-centric multilingual machine translation," *Journal of Machine Learning Research*, vol. 22, no. 107, pp. 1–48, 2021.

[10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of KDD*, 2016, pp. 785–794.

[11] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988.

[12] R. Mayer, *Multimedia Learning*, 2nd ed. Cambridge, UK: Cambridge University Press, 2009.

[13] A. Collins, J. S. Brown, and S. E. Newman, "Cognitive apprenticeship: Teaching the crafts of reading, writing, and mathematics," in *Knowing, Learning, and Instruction*, L. B. Resnick, Ed. Hillsdale, NJ: Lawrence Erlbaum, 1989, pp. 453–494.

[14] V. Rastvorov, *ELA: English Language Assistant*. Research thesis, Munster Technological University Cork, 2025.

[15] N. Ahmad and V. Rastvorov, *ODE-Segment Sequence Forecasting: Learning Weather Pattern Grammar from Single-Station Observations*. Technical report, Munster Technological University Cork, 2026. Available: https://github.com/welsol21/weather_forecast

[16] V. Rastvorov, *ELA: English Language Assistant — Source Repository*. 2026. Available: https://github.com/welsol21/FYP_LLM

[17] Universal Dependencies, *UD English-EWT*. Available: https://universaldependencies.org/treebanks/en_ewt/index.html

[18] Universal Dependencies, *UD English-GUM*. Available: https://universaldependencies.org/treebanks/en_gum/index.html

[19] Tatoeba Contributors, *Tatoeba Corpus*. Available: https://tatoeba.org/

[20] Wikinews Contributors, *Wikinews*. Available: https://en.wikinews.org/

[21] Simple Wiktionary Contributors, *Simple English Wiktionary*. Available: https://simple.wiktionary.org/

[22] B. S. Azar, *Basic English Grammar*, 2nd ed. White Plains, NY: Pearson Education, 1996.

[23] K. Börjars and K. Burridge, *Introducing English Grammar*, 2nd ed. London, UK: Hodder Education, 2010.

[24] M. Lester, *English Grammar Drills*. New York, NY: McGraw-Hill, 2009.

[25] A. S. Calude and L. Bauer, *Mysteries of English Grammar: A Guide to Complexities of the English Language*. London, UK: Reaktion Books, 2022.

[26] G. Yule, *Explaining English Grammar*. Oxford, UK: Oxford University Press, 1998.

[27] P. Simon, *The Grammaring Guide to English Grammar*. 2013.

[28] G. Woods, *English Grammar For Dummies*. Hoboken, NJ: Wiley, 2017.

[29] P. Dutwin, *English Grammar Demystified*. New York, NY: McGraw-Hill, 2010.

[30] S. Brehe, *Brehe's Grammar Anatomy*. 2019.

[31] R. Thiessen, *English Grammar for Academic Purposes*. 2025.

[32] V. Rastvorov, *ELA Note-ID Dynamics Report*. Repository report, 2026. Available: https://github.com/welsol21/FYP_LLM/blob/main/docs/reports/ela_note_id_dynamics_report_2026-04-30.md

[33] V. Rastvorov, *Contract-Input Raw-Note Dataset Stats*. Repository artifact, 2026. Available: https://github.com/welsol21/FYP_LLM/blob/main/data/processed_sentence_seed/projection_external_sentence_contract_v2_note_first_balanced_exact5_cap104_v3/stats.json

[34] V. Rastvorov, *Mixed Paired-Template Dataset Stats (v45)*. Repository artifact, 2026. Available: https://github.com/welsol21/FYP_LLM/blob/main/data/processed_sentence_seed/seed_preserving_sentence_dataset_v45_paired_template_sentence_nodes_contractfix4_v1/stats.json